

VL-03: Registermaschinen

(Berechenbarkeit und Komplexität, WS 2024)

Peter Rossmanith

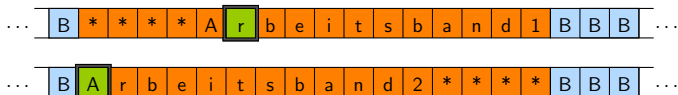
WS 2024, RWTH

Wiederholung

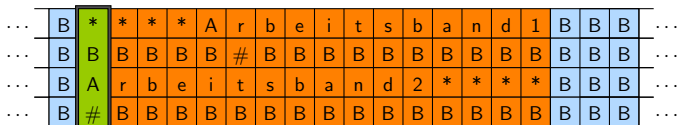
Wdh.: k -Band- vs 1-Band-TM

Satz

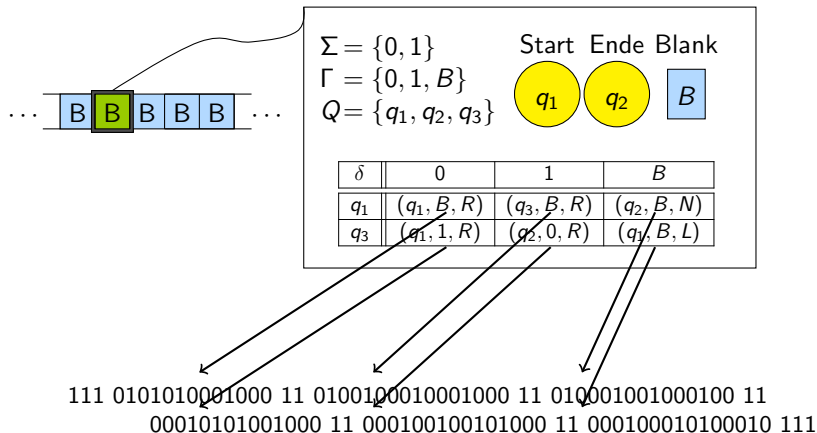
Eine k -Band-TM M mit Zeitbedarf $t(n)$ und Platzbedarf $s(n)$ kann von einer 1-Band-TM M' mit Zeitbedarf $O(t^2(n))$ und Platzbedarf $O(s(n))$ simuliert werden.



Simuliert durch

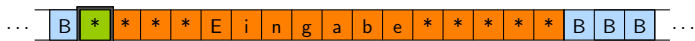


Wdh.: Gödelnummer $\langle M \rangle$

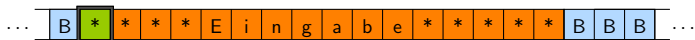


Wdh.: Universelle TM

Simulierte Turingmaschine M



Initialisierung der universellen Maschine U



Laufzeit der universellen TM

- Bei Eingabe $\langle M \rangle w$ simuliert U die TM M auf Wort w .
- Jeder Schritt von M wird dabei von U in $f(|\langle M \rangle|)$ Zeit simuliert.
- Wenn $|\langle M \rangle|$ als Konstante angesehen wird, so simuliert U die TM M mit konstantem Zeit- und Platzverlust.

Alonzo Church und Alan Turing formulierten in den 1930er Jahren die folgende These:

Church-Turing These

Die Klasse der TM-berechenbaren Funktionen stimmt mit der Klasse der “intuitiv berechenbaren” Funktionen überein.

Sprachliche Übereinkunft für den Rest der Vorlesung:
berechenbare Funktion = TM-berechenbare Funktion = rekursive Funktion
entscheidbare Sprache = TM-entscheidbare Sprache = rekursive Sprache

Vorlesung VL-03

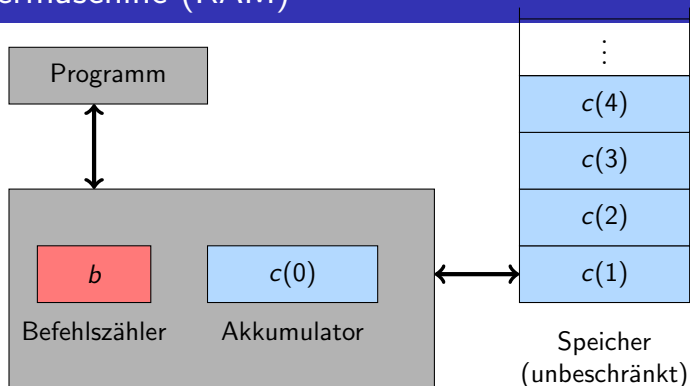
Registermaschinen

- Registermaschinen (RAMs)
- Simulation von RAM durch TM
- Simulation von TM durch RAM

- Collatz Problem

Registermaschinen (RAM)

Registermaschine (RAM)



Befehlssatz:

LOAD, STORE, ADD, SUB, MULT, DIV
INDLOAD, INDSTORE, INDADD, INDSUB, INDMULT, INDDIV
CLOAD, CADD, CSUB, CMULT, CDIV
GOTO, IF $c(0)?x$ THEN GOTO j (mit $? \in \{=, <, \leq, >, \geq\}$)
END

RAM: Syntax und Semantik der Befehle (1)

Syntax	Bewirkte Veränderung	Befehlszähler
LOAD i	$c(0) := c(i)$	$b := b + 1$
CLOAD i	$c(0) := i$	$b := b + 1$
INDLOAD i	$c(0) := c(c(i))$	$b := b + 1$

RAM: Syntax und Semantik der Befehle (1)

Syntax	Bewirkte Veränderung	Befehlszähler
LOAD i	$c(0) := c(i)$	$b := b + 1$
CLOAD i	$c(0) := i$	$b := b + 1$
INDLOAD i	$c(0) := c(c(i))$	$b := b + 1$
STORE i	$c(i) := c(0)$	$b := b + 1$
INDSTORE i	$c(c(i)) := c(0)$	$b := b + 1$

RAM: Syntax und Semantik der Befehle (1)

Syntax	Bewirkte Veränderung	Befehlszähler
LOAD i	$c(0) := c(i)$	$b := b + 1$
CLOAD i	$c(0) := i$	$b := b + 1$
INDLOAD i	$c(0) := c(c(i))$	$b := b + 1$
STORE i	$c(i) := c(0)$	$b := b + 1$
INDSTORE i	$c(c(i)) := c(0)$	$b := b + 1$
ADD i	$c(0) := c(0) + c(i)$	$b := b + 1$
CADD i	$c(0) := c(0) + i$	$b := b + 1$
INDADD i	$c(0) := c(0) + c(c(i))$	$b := b + 1$

RAM: Syntax und Semantik der Befehle (1)

Syntax	Bewirkte Veränderung	Befehlszähler
LOAD i	$c(0) := c(i)$	$b := b + 1$
CLOAD i	$c(0) := i$	$b := b + 1$
INDLOAD i	$c(0) := c(c(i))$	$b := b + 1$
STORE i	$c(i) := c(0)$	$b := b + 1$
INDSTORE i	$c(c(i)) := c(0)$	$b := b + 1$
ADD i	$c(0) := c(0) + c(i)$	$b := b + 1$
CADD i	$c(0) := c(0) + i$	$b := b + 1$
INDADD i	$c(0) := c(0) + c(c(i))$	$b := b + 1$
SUB i	$c(0) := c(0) - c(i)$	$b := b + 1$
CSUB i	$c(0) := c(0) - i$	$b := b + 1$
INDSUB i	$c(0) := c(0) - c(c(i))$	$b := b + 1$

RAM: Syntax und Semantik der Befehle (1)

Syntax	Bewirkte Veränderung	Befehlszähler
LOAD i	$c(0) := c(i)$	$b := b + 1$
CLOAD i	$c(0) := i$	$b := b + 1$
INDLOAD i	$c(0) := c(c(i))$	$b := b + 1$
STORE i	$c(i) := c(0)$	$b := b + 1$
INDSTORE i	$c(c(i)) := c(0)$	$b := b + 1$
ADD i	$c(0) := c(0) + c(i)$	$b := b + 1$
CADD i	$c(0) := c(0) + i$	$b := b + 1$
INDADD i	$c(0) := c(0) + c(c(i))$	$b := b + 1$
SUB i	$c(0) := c(0) - c(i)$	$b := b + 1$
CSUB i	$c(0) := c(0) - i$	$b := b + 1$
INDSUB i	$c(0) := c(0) - c(c(i))$	$b := b + 1$
MULT i	$c(0) := c(0) * c(i)$	$b := b + 1$
CMULT i	$c(0) := c(0) * i$	$b := b + 1$
INDMULT i	$c(0) := c(0) * c(c(i))$	$b := b + 1$

RAM: Syntax und Semantik der Befehle (1)

Syntax	Bewirkte Veränderung	Befehlszähler
LOAD i	$c(0) := c(i)$	$b := b + 1$
CLOAD i	$c(0) := i$	$b := b + 1$
INDLOAD i	$c(0) := c(c(i))$	$b := b + 1$
STORE i	$c(i) := c(0)$	$b := b + 1$
INDSTORE i	$c(c(i)) := c(0)$	$b := b + 1$
ADD i	$c(0) := c(0) + c(i)$	$b := b + 1$
CADD i	$c(0) := c(0) + i$	$b := b + 1$
INDADD i	$c(0) := c(0) + c(c(i))$	$b := b + 1$
SUB i	$c(0) := c(0) - c(i)$	$b := b + 1$
CSUB i	$c(0) := c(0) - i$	$b := b + 1$
INDSUB i	$c(0) := c(0) - c(c(i))$	$b := b + 1$
MULT i	$c(0) := c(0) * c(i)$	$b := b + 1$
CMULT i	$c(0) := c(0) * i$	$b := b + 1$
INDMULT i	$c(0) := c(0) * c(c(i))$	$b := b + 1$
DIV i	$c(0) := \lfloor c(0)/c(i) \rfloor$	$b := b + 1$
CDIV i	$c(0) := \lfloor c(0)/i \rfloor$	$b := b + 1$
INDDIV i	$c(0) := \lfloor c(0)/c(c(i)) \rfloor$	$b := b + 1$

DIV, CDIV, INDDIV: Bei Division durch 0 ist das Ergebnis 0

RAM: Syntax und Semantik der Befehle (2)

Syntax	Bewirkte Veränderung	Befehlszähler
GOTO j	—	$b := j$
IF $c(0) = x$ GOTO j	—	$b := \begin{cases} j & \text{falls } c(0) = x \\ b + 1 & \text{sonst} \end{cases}$
IF $c(0) < x$ GOTO j	—	$b := \begin{cases} j & \text{falls } c(0) < x \\ b + 1 & \text{sonst} \end{cases}$
IF $c(0) \leq x$ GOTO j	—	$b := \begin{cases} j & \text{falls } c(0) \leq x \\ b + 1 & \text{sonst} \end{cases}$
IF $c(0) > x$ GOTO j	—	$b := \begin{cases} j & \text{falls } c(0) > x \\ b + 1 & \text{sonst} \end{cases}$
IF $c(0) \geq x$ GOTO j	—	$b := \begin{cases} j & \text{falls } c(0) \geq x \\ b + 1 & \text{sonst} \end{cases}$
END	Ende der Berechnung	—

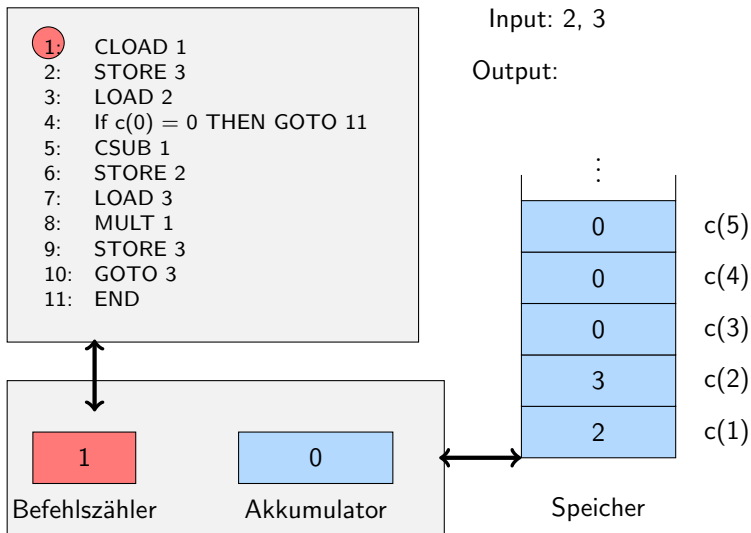
RAM: Funktionsweise

- Der Speicher der RAM ist unbeschränkt und besteht aus dem Akkumulator $c(0)$ und aus den Registern $c(1), c(2), c(3), \dots$
- Die Inhalte der Register sind natürliche Zahlen, die beliebig gross werden können.

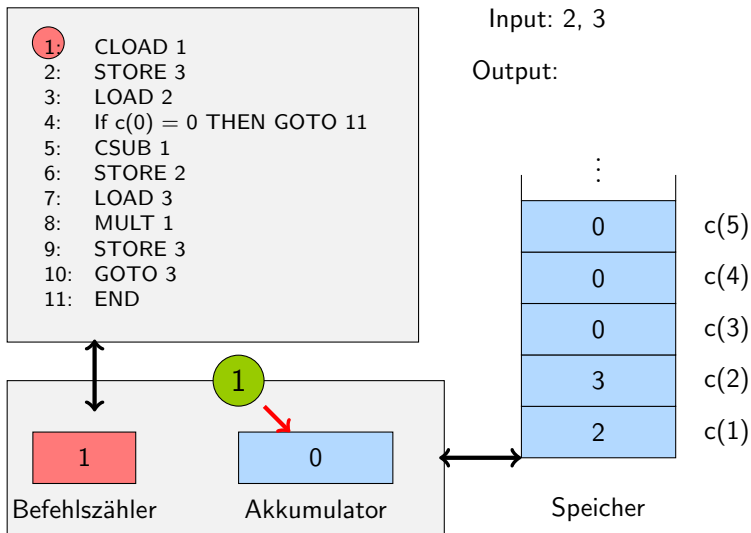
- Der Speicher der RAM ist unbeschränkt und besteht aus dem Akkumulator $c(0)$ und aus den Registern $c(1), c(2), c(3), \dots$
- Die Inhalte der Register sind natürliche Zahlen, die beliebig gross werden können.
- Die Eingabe besteht ebenfalls aus natürlichen Zahlen, die zu Beginn in den ersten paar Registern abgespeichert sind.
- Alle anderen Register sind mit 0 initialisiert.
- Der Befehlszähler startet mit dem Wert 1. Ausgeführt wird jeweils der Befehl in derjenigen Zeile, auf die der Befehlszähler verweist.

- Der Speicher der RAM ist unbeschränkt und besteht aus dem Akkumulator $c(0)$ und aus den Registern $c(1), c(2), c(3), \dots$
- Die Inhalte der Register sind natürliche Zahlen, die beliebig gross werden können.
- Die Eingabe besteht ebenfalls aus natürlichen Zahlen, die zu Beginn in den ersten paar Registern abgespeichert sind.
- Alle anderen Register sind mit 0 initialisiert.
- Der Befehlszähler startet mit dem Wert 1. Ausgeführt wird jeweils der Befehl in derjenigen Zeile, auf die der Befehlszähler verweist.
- Die Rechnung stoppt, sobald der Befehl END erreicht ist.
- Die Ausgabe befindet sich nach dem Stoppen in den ersten paar Registern.

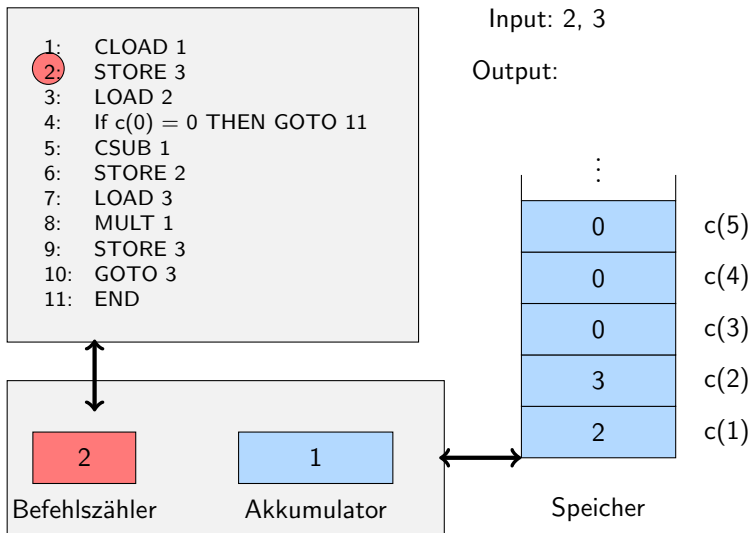
RAM: Beispielprogramm



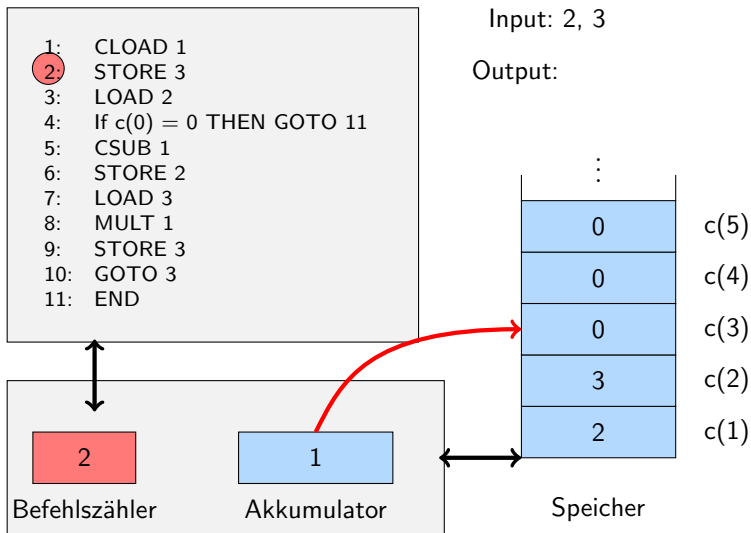
RAM: Beispielprogramm



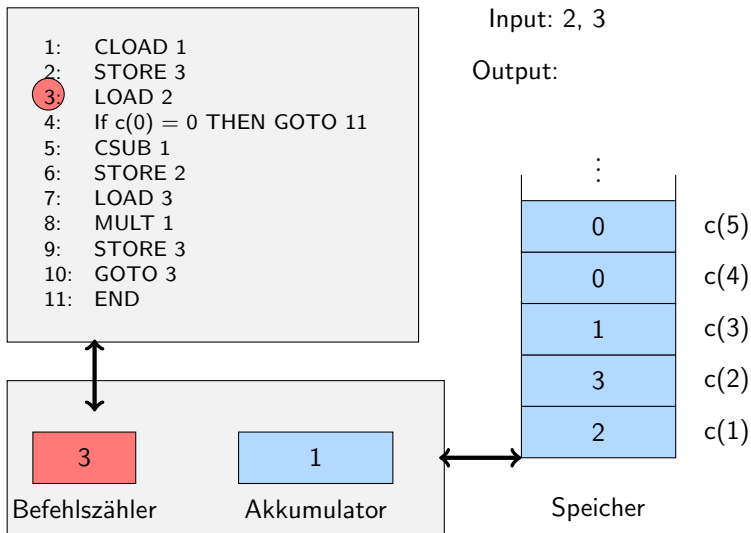
RAM: Beispielprogramm



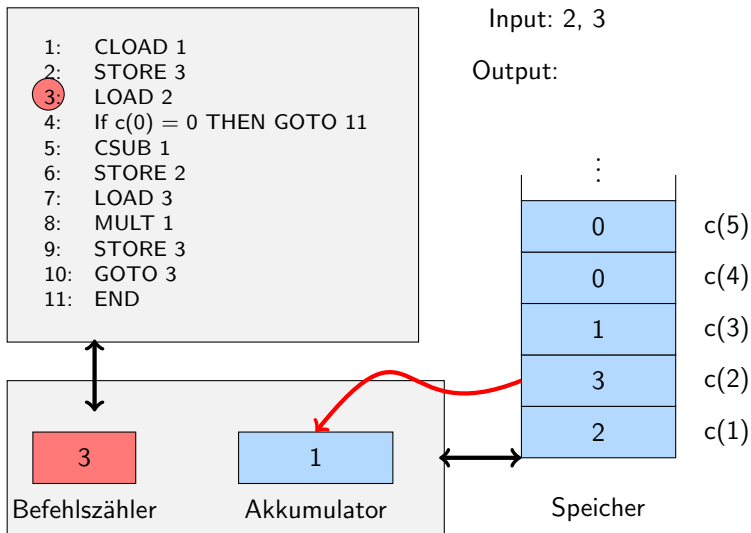
RAM: Beispielprogramm



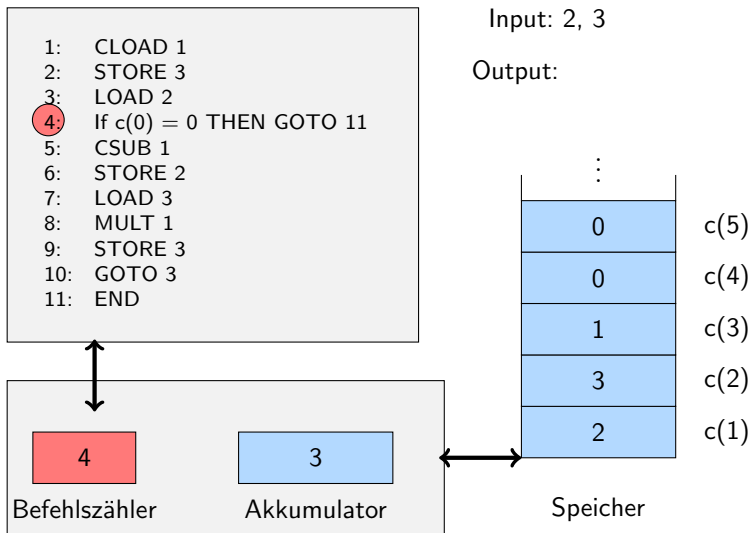
RAM: Beispielprogramm



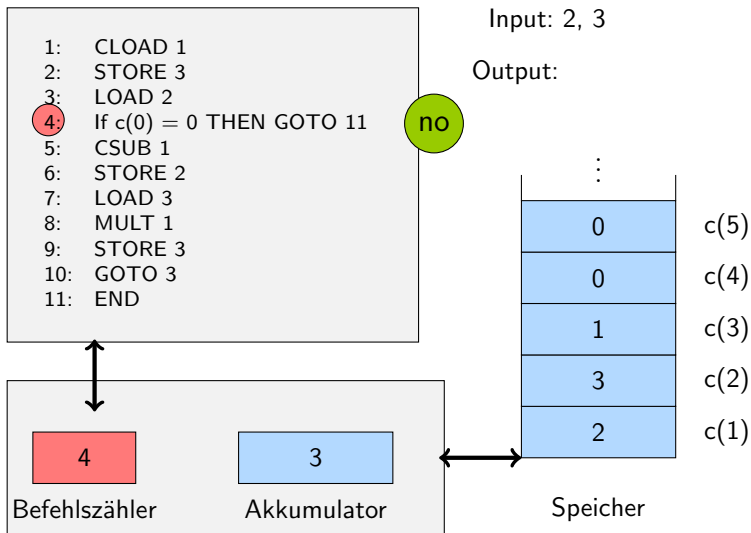
RAM: Beispielprogramm



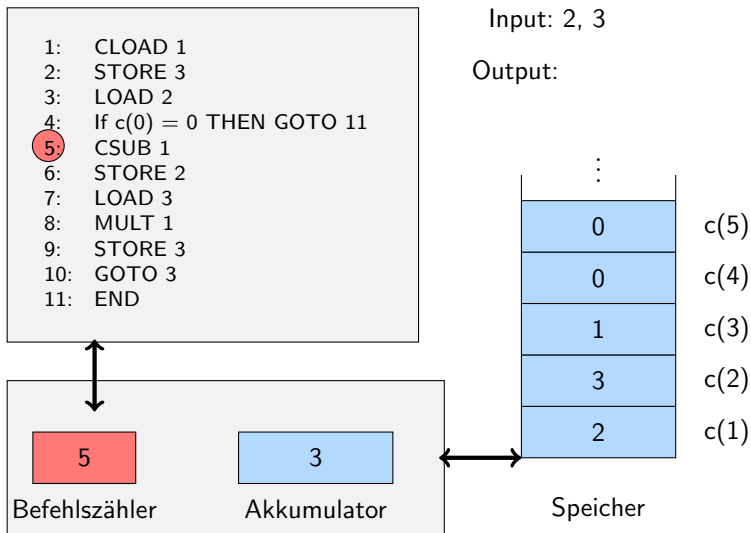
RAM: Beispielprogramm



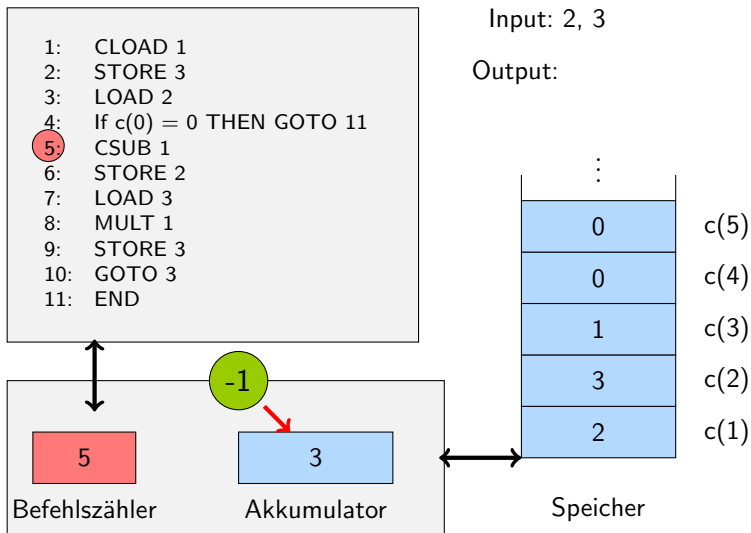
RAM: Beispielprogramm



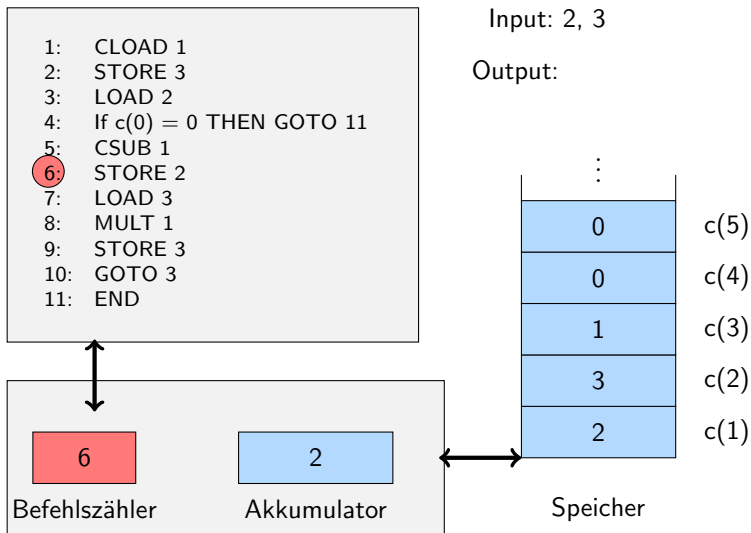
RAM: Beispielprogramm



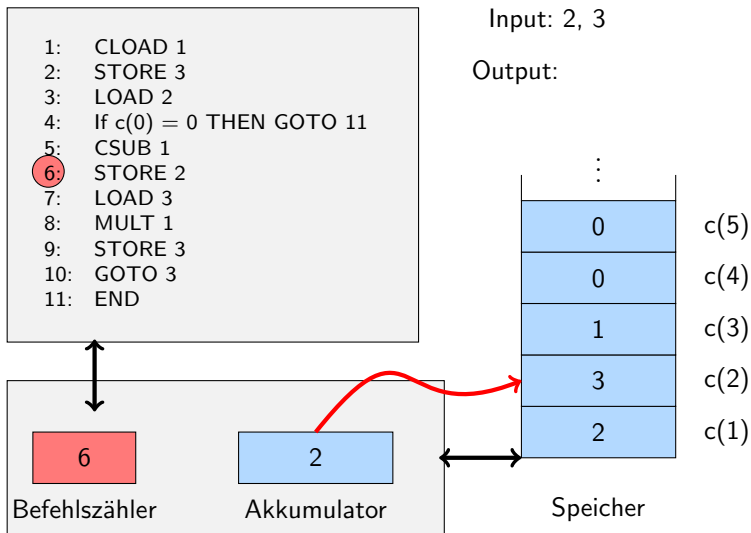
RAM: Beispielprogramm



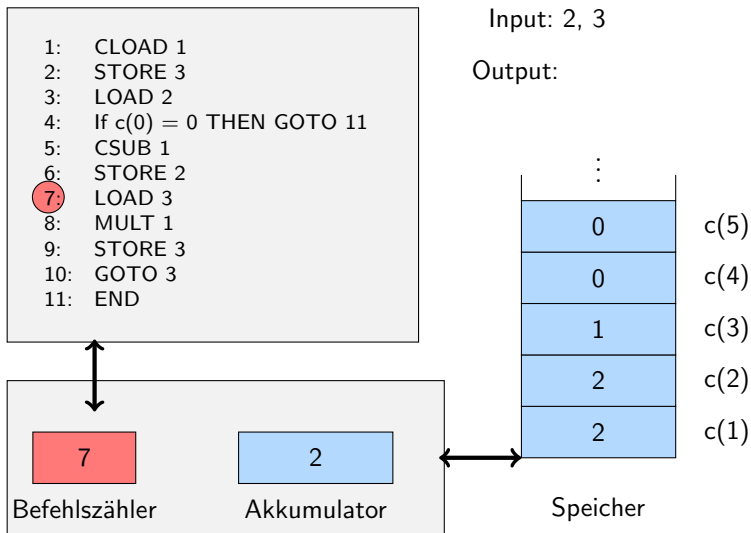
RAM: Beispielprogramm



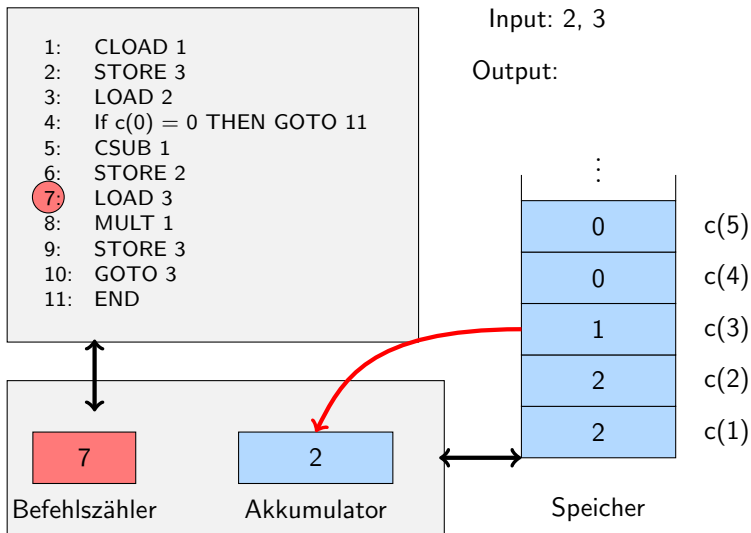
RAM: Beispielprogramm



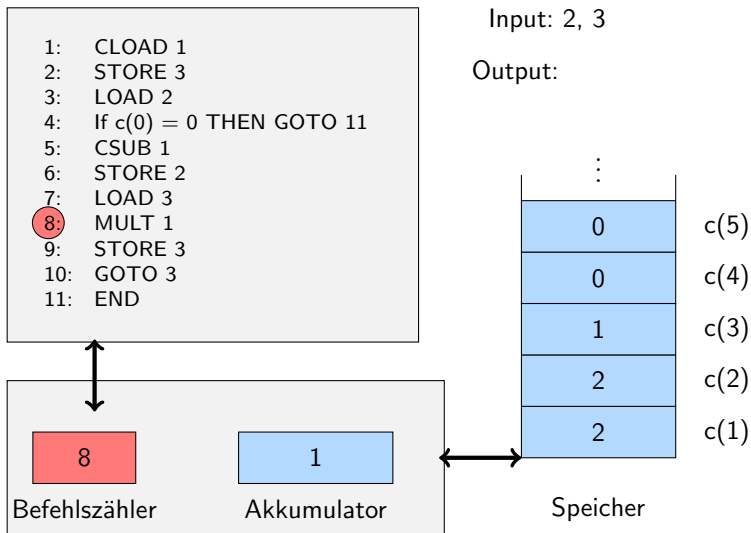
RAM: Beispielprogramm



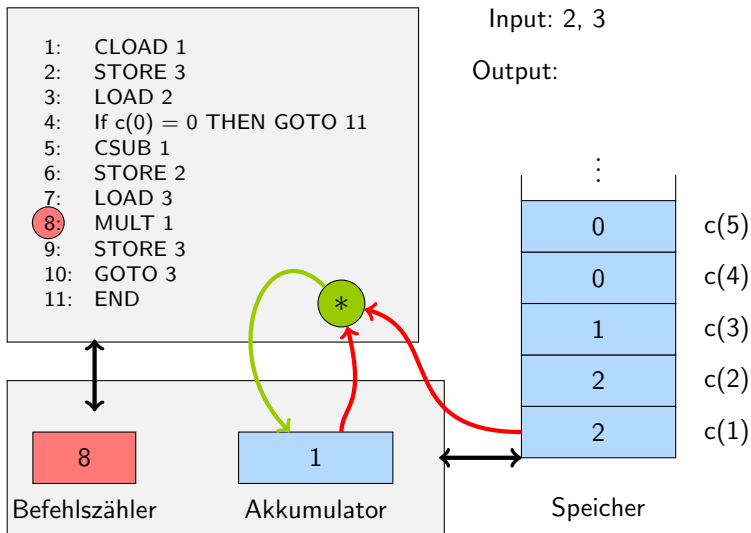
RAM: Beispielprogramm



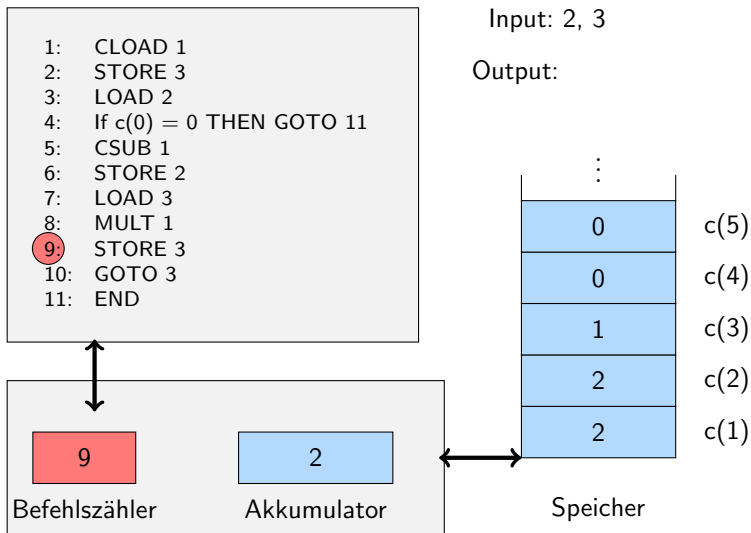
RAM: Beispielprogramm



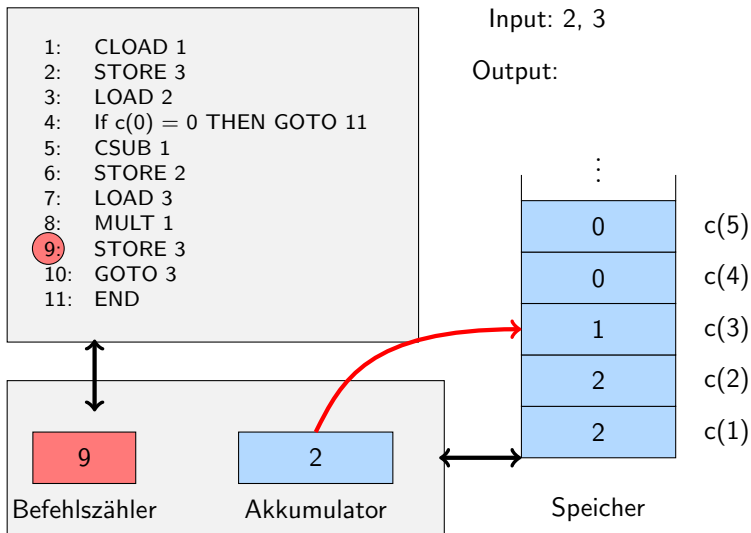
RAM: Beispielprogramm



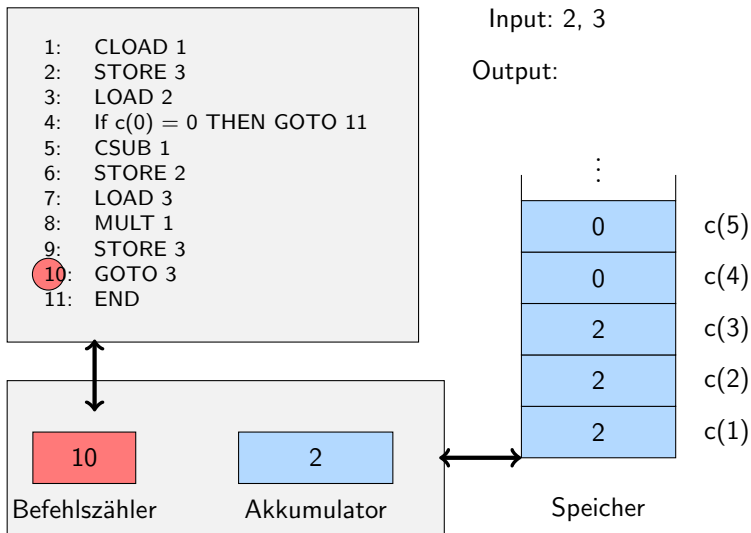
RAM: Beispielprogramm



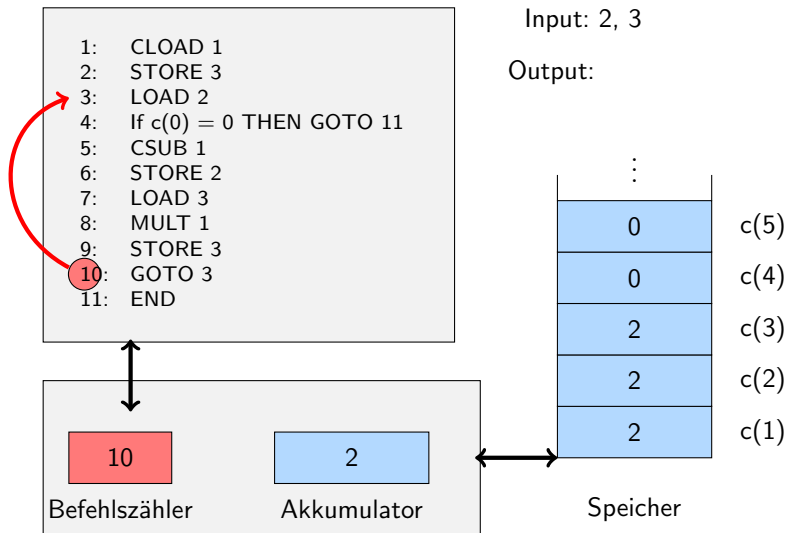
RAM: Beispielprogramm



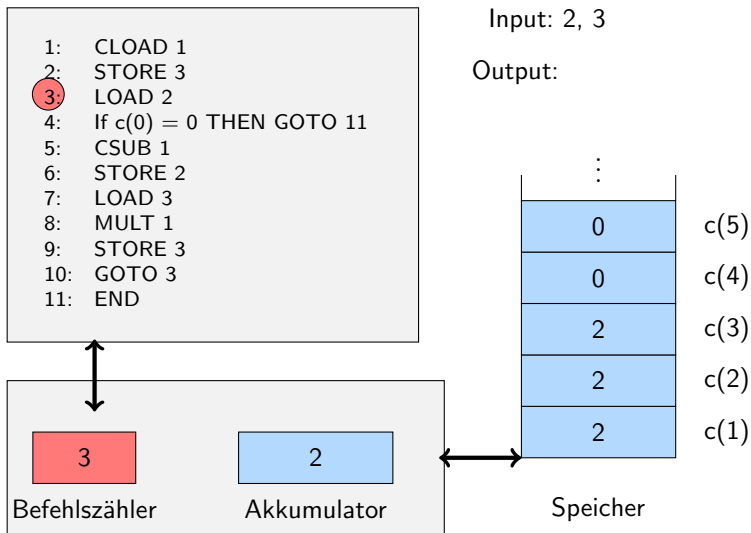
RAM: Beispielprogramm



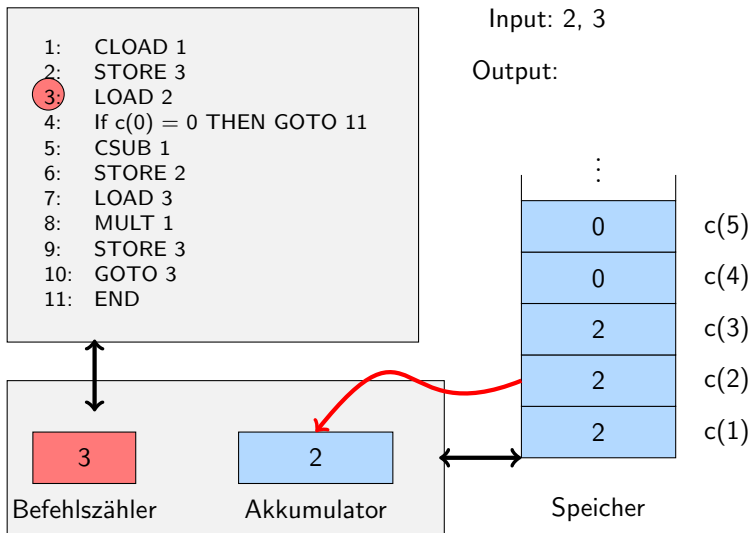
RAM: Beispielprogramm



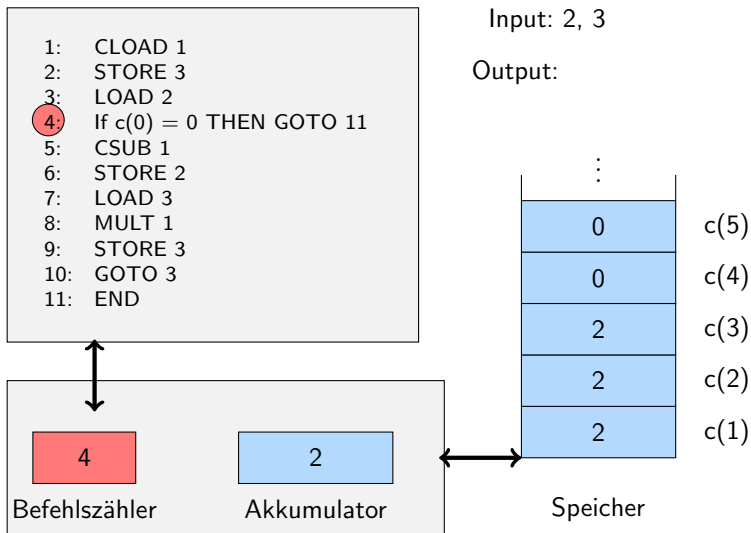
RAM: Beispielprogramm



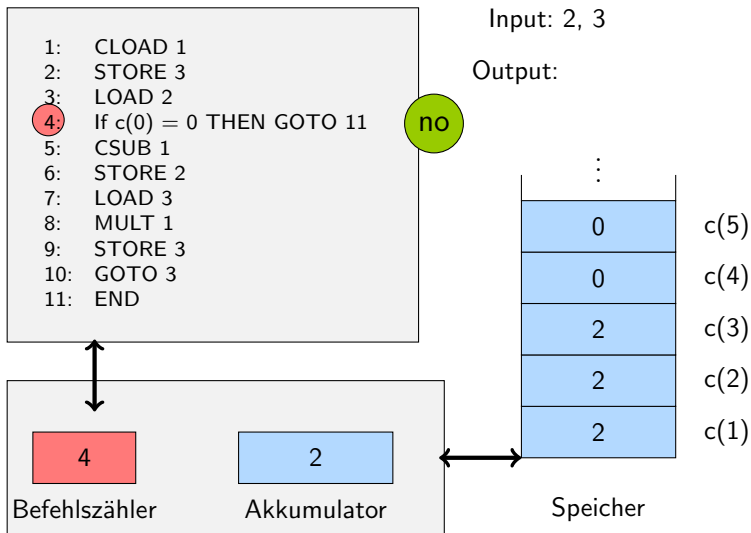
RAM: Beispielprogramm



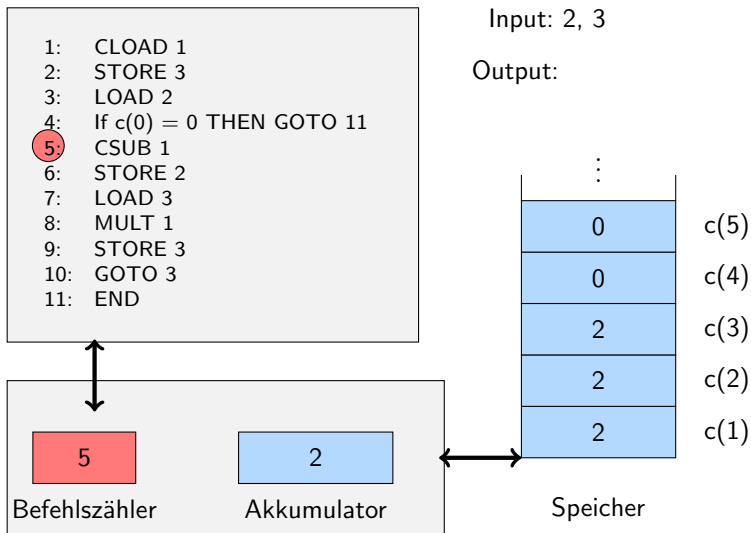
RAM: Beispielprogramm



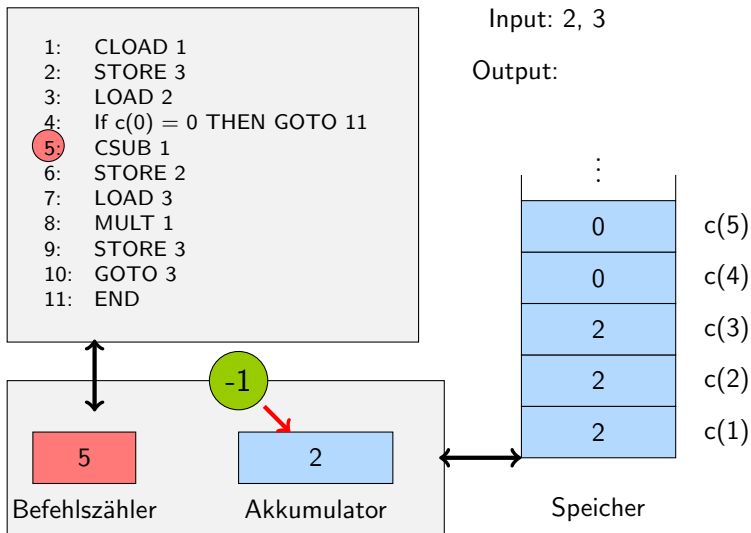
RAM: Beispielprogramm



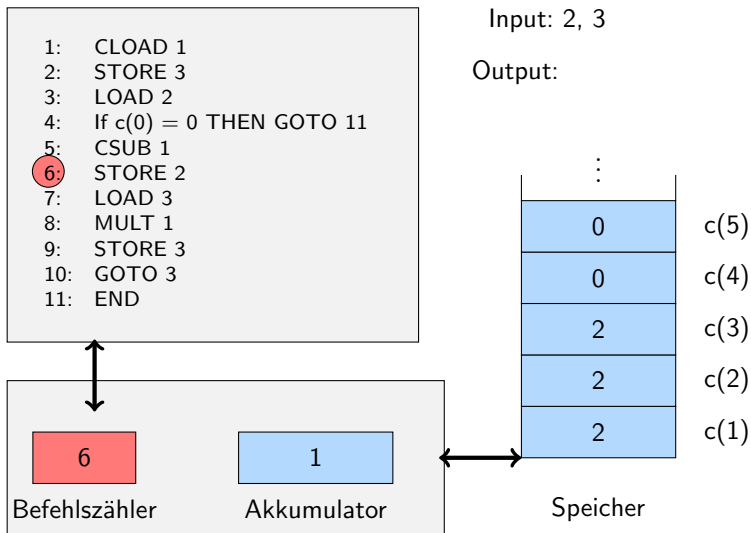
RAM: Beispielprogramm



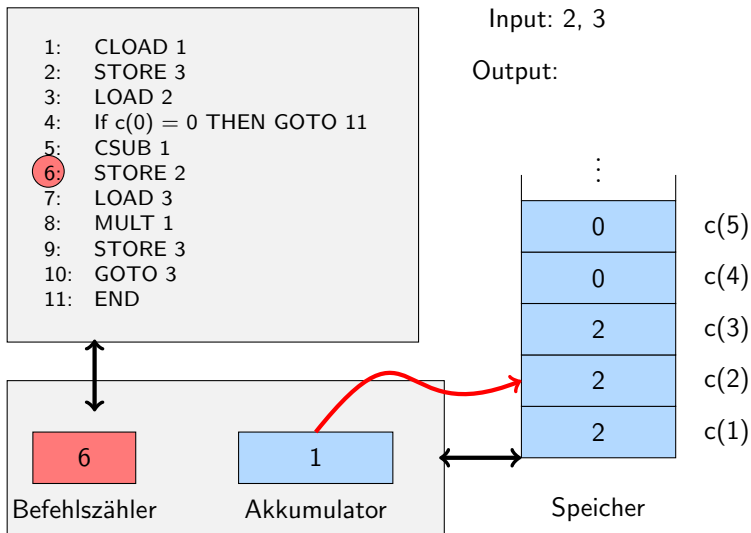
RAM: Beispielprogramm



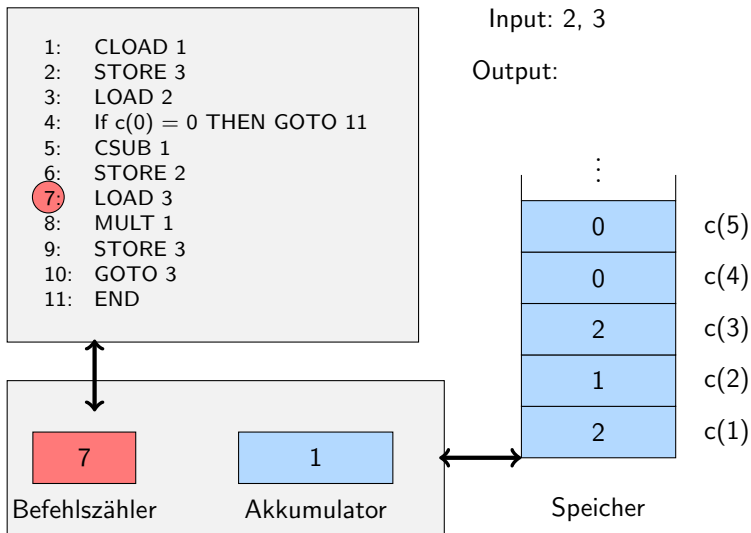
RAM: Beispielprogramm



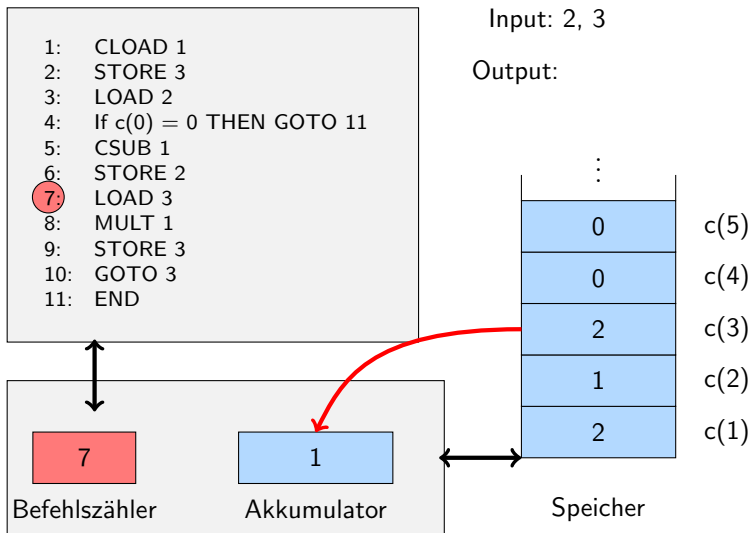
RAM: Beispielprogramm



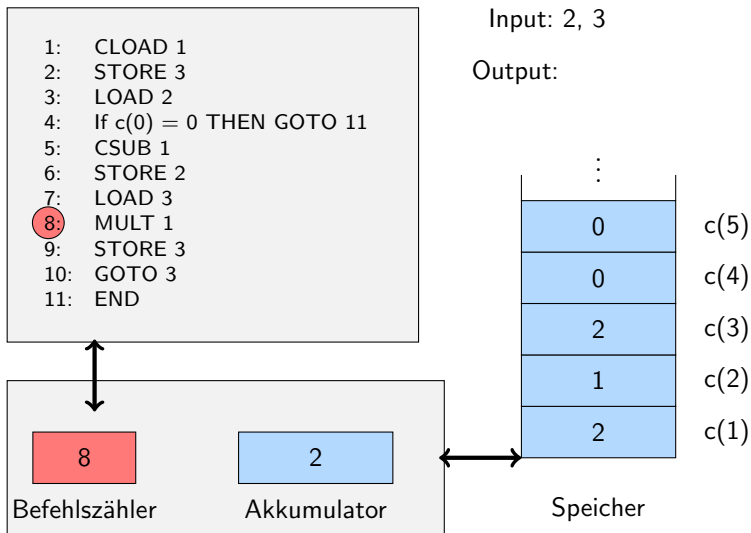
RAM: Beispielprogramm



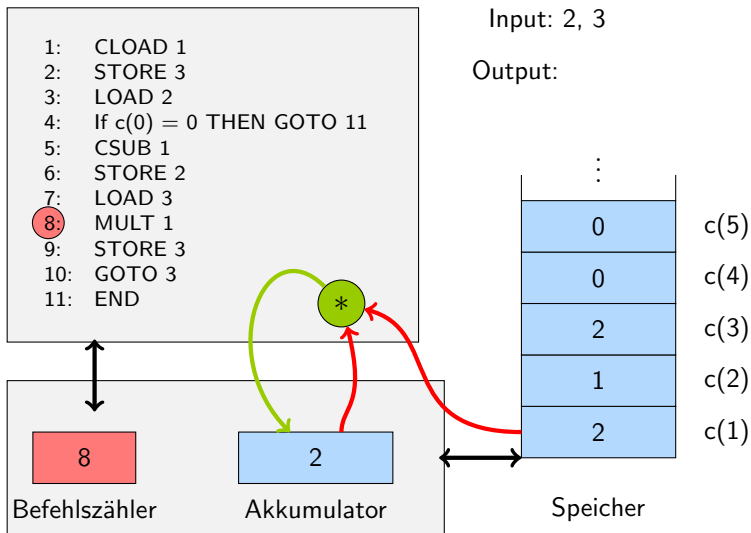
RAM: Beispielprogramm



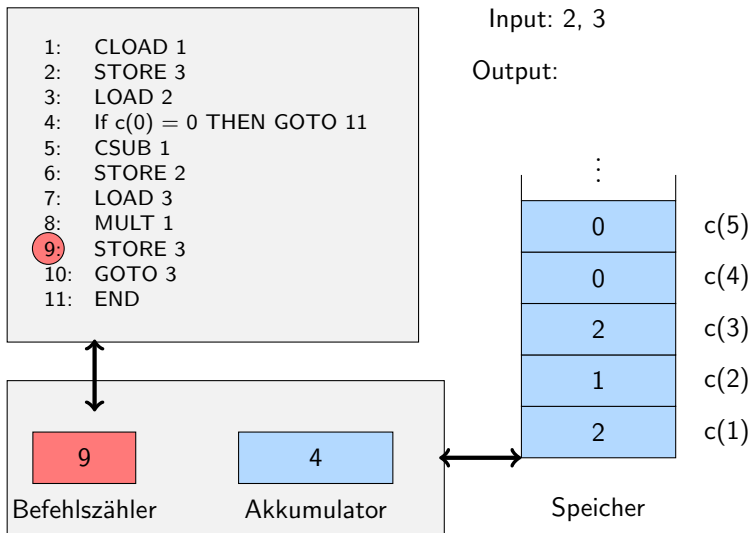
RAM: Beispielprogramm



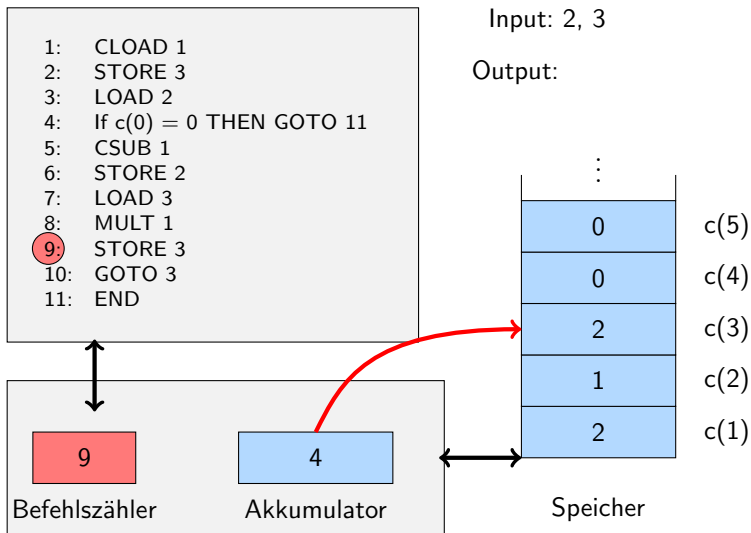
RAM: Beispielprogramm



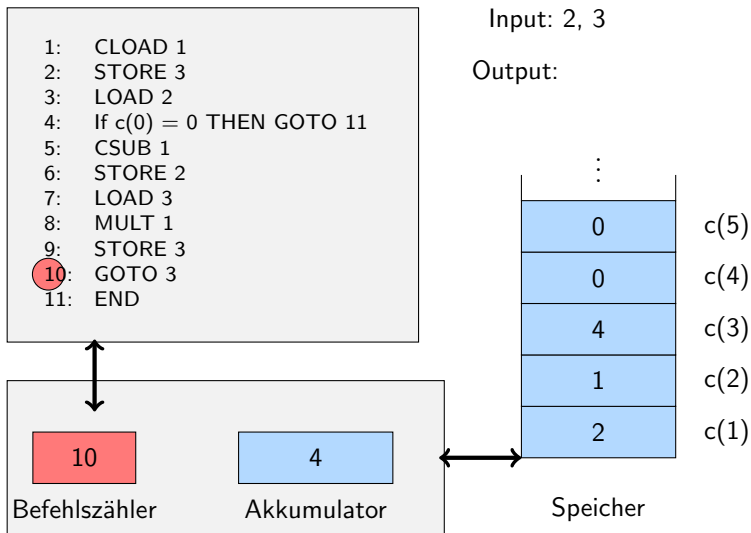
RAM: Beispielprogramm



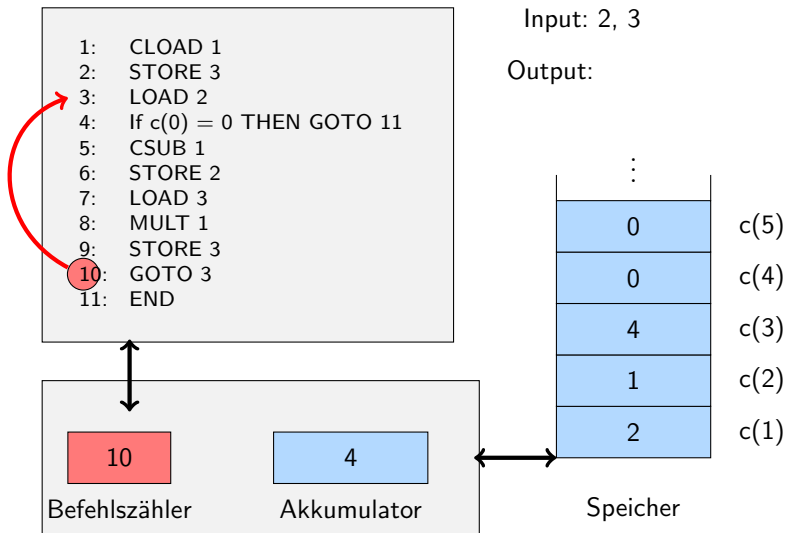
RAM: Beispielprogramm



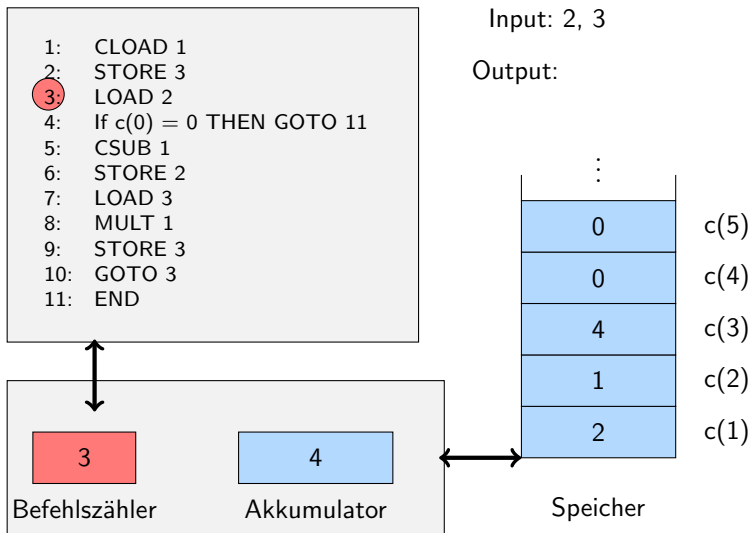
RAM: Beispielprogramm



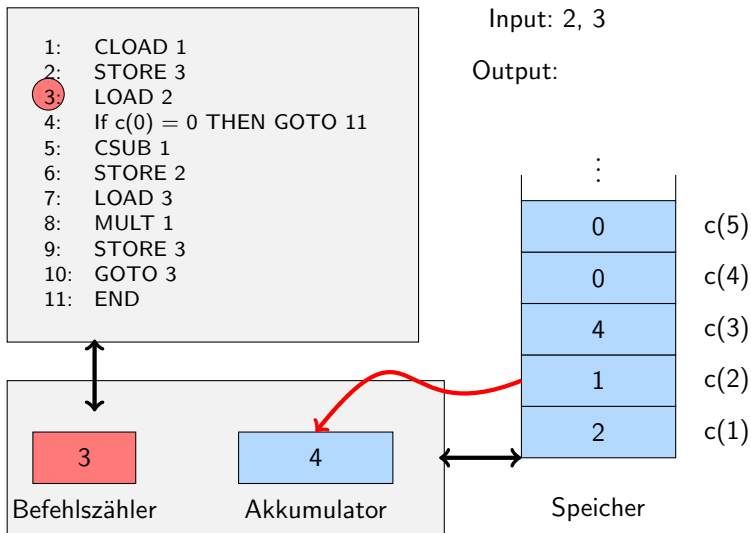
RAM: Beispielprogramm



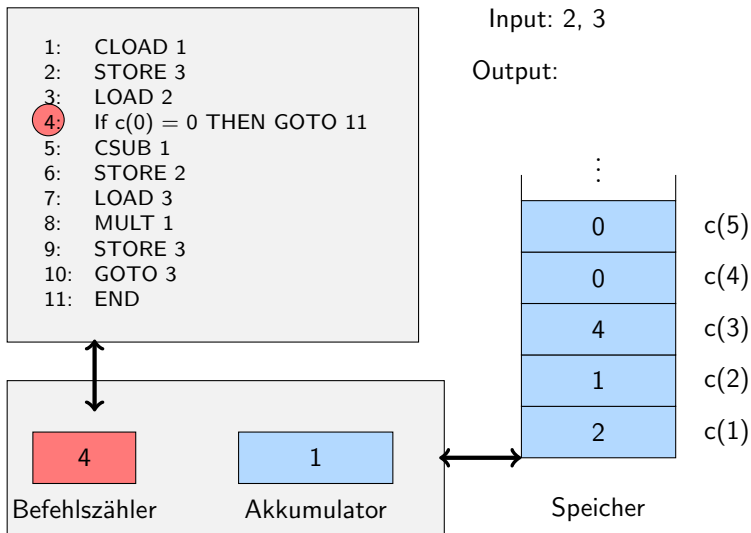
RAM: Beispielprogramm



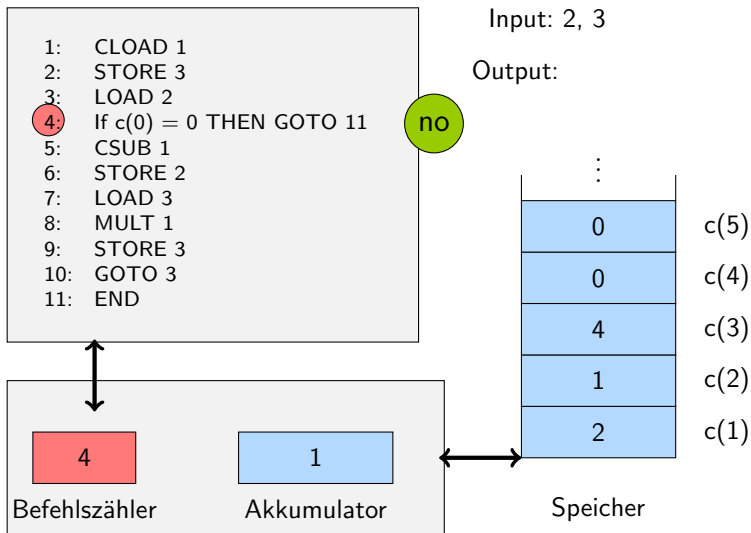
RAM: Beispielprogramm



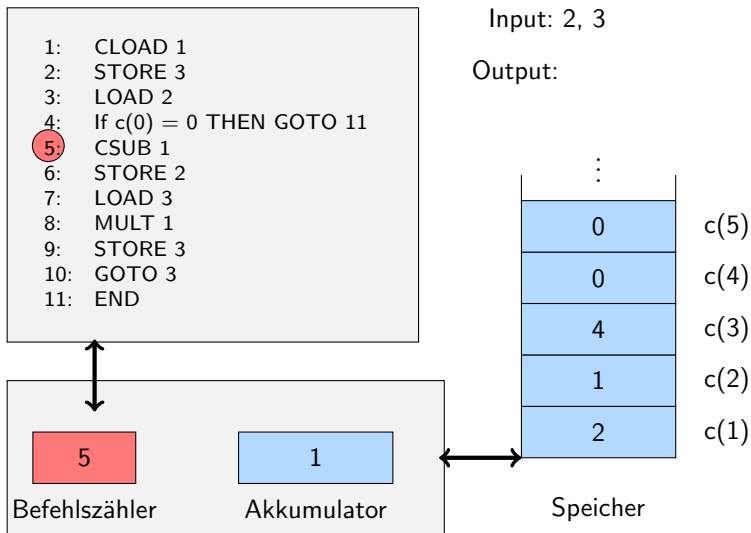
RAM: Beispielprogramm



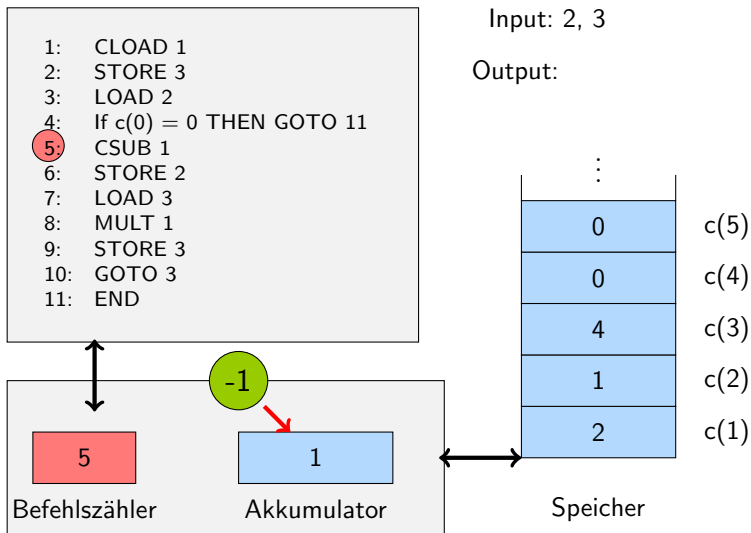
RAM: Beispielprogramm



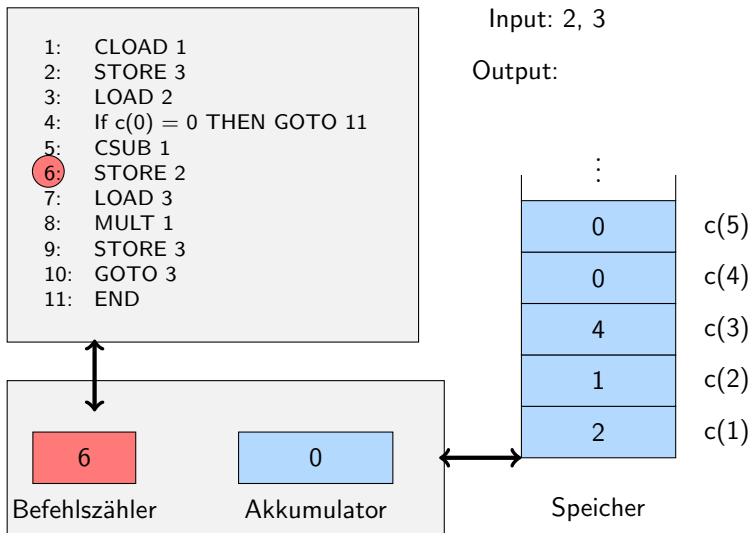
RAM: Beispielprogramm



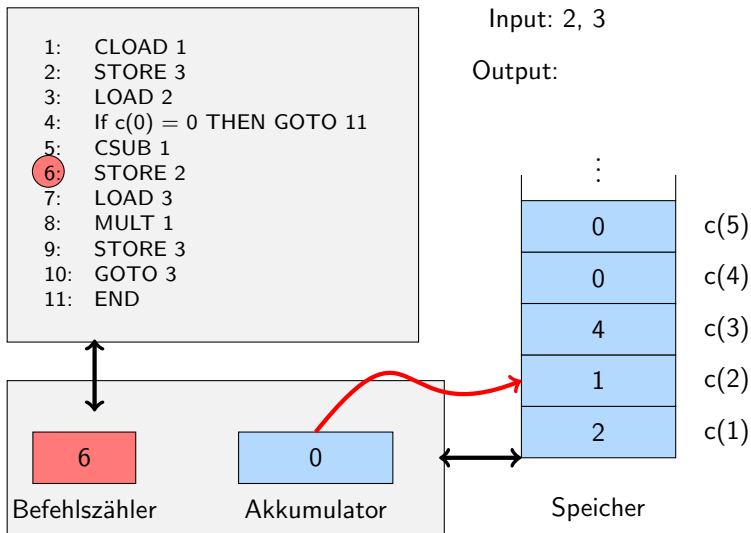
RAM: Beispielprogramm



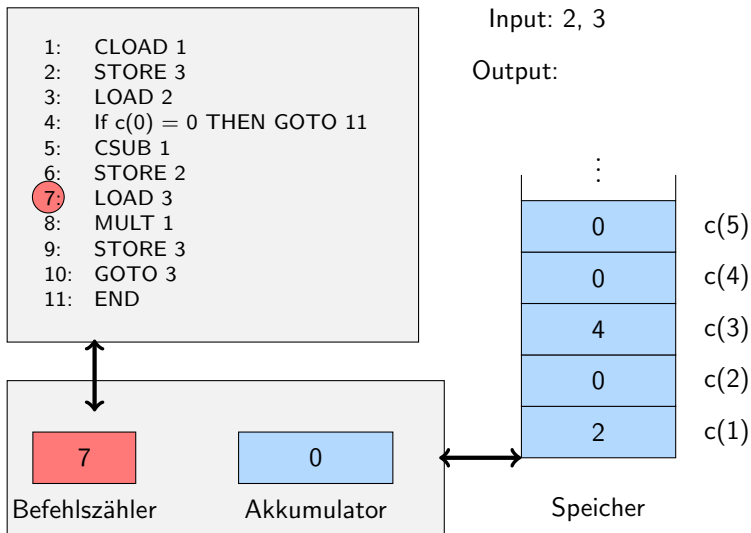
RAM: Beispielprogramm



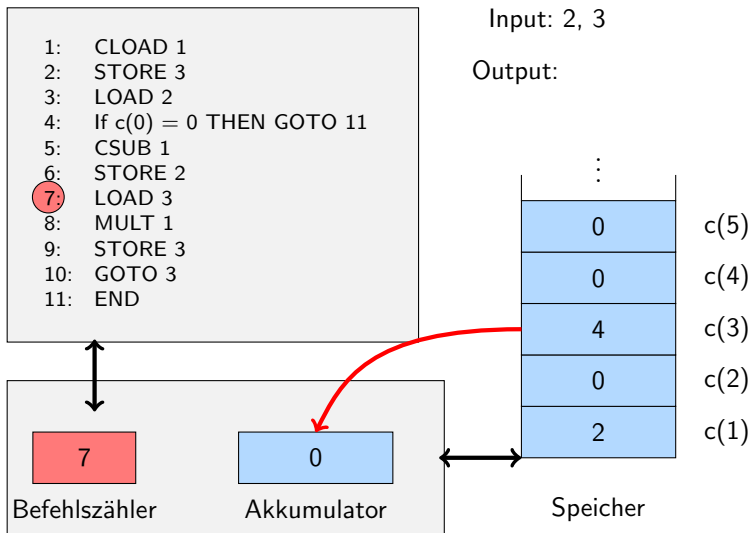
RAM: Beispielprogramm



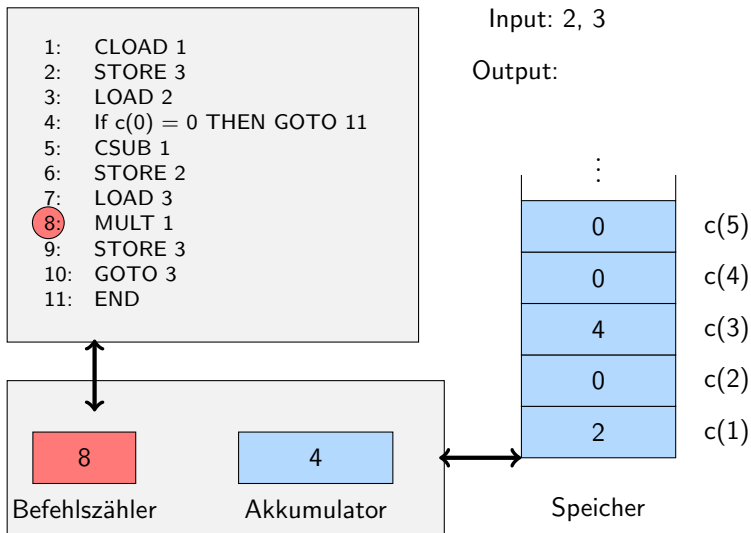
RAM: Beispielprogramm



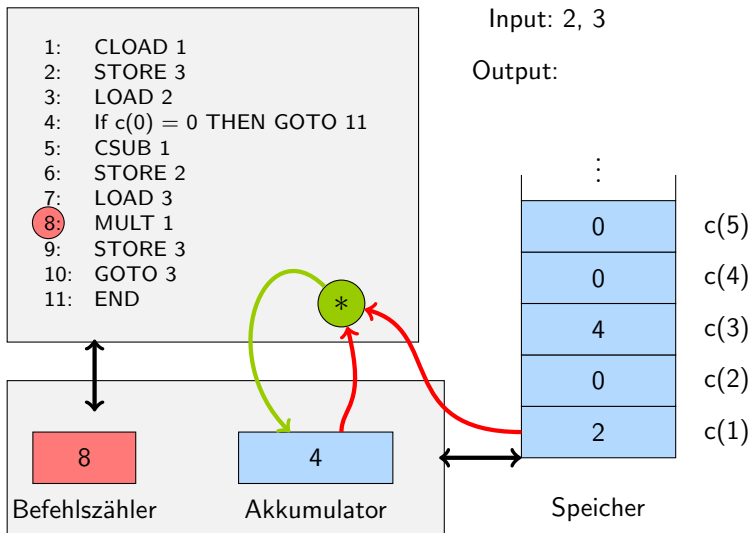
RAM: Beispielprogramm



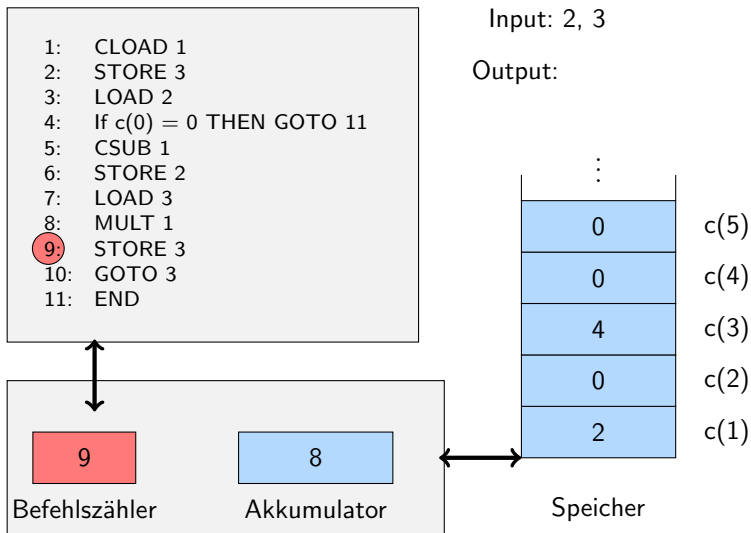
RAM: Beispielprogramm



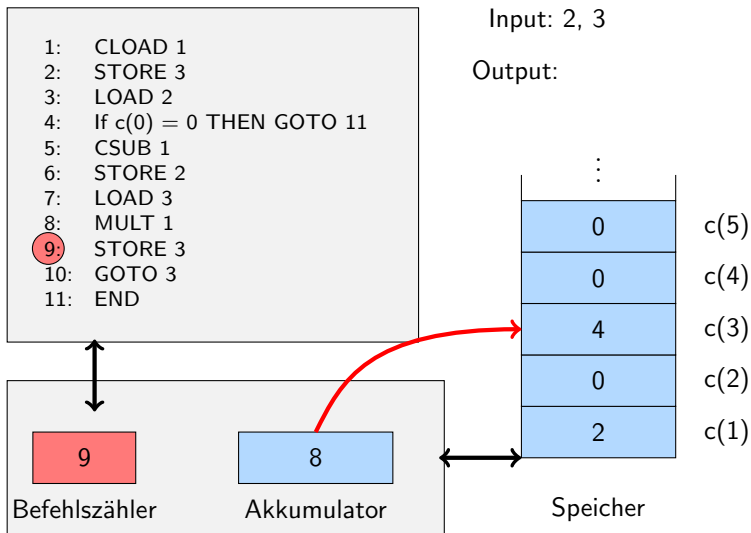
RAM: Beispielprogramm



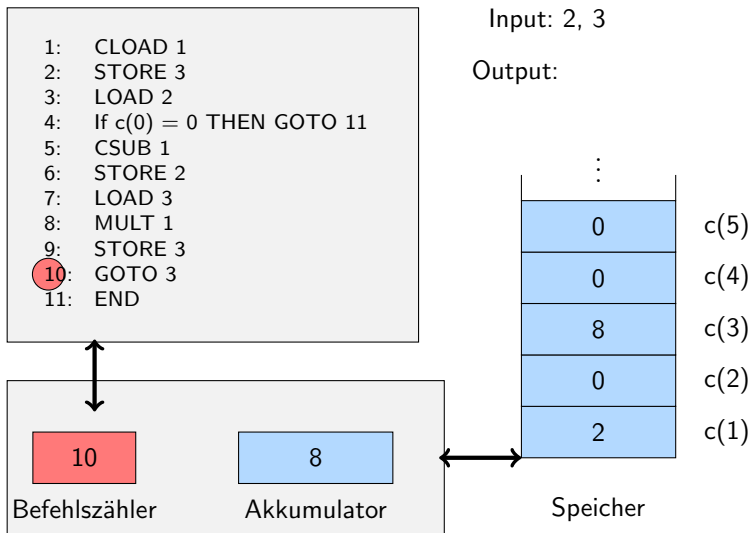
RAM: Beispielprogramm



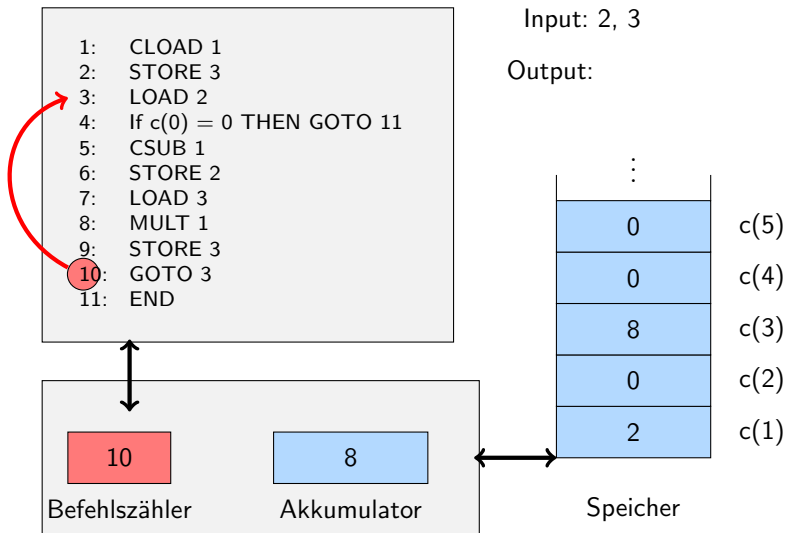
RAM: Beispielprogramm



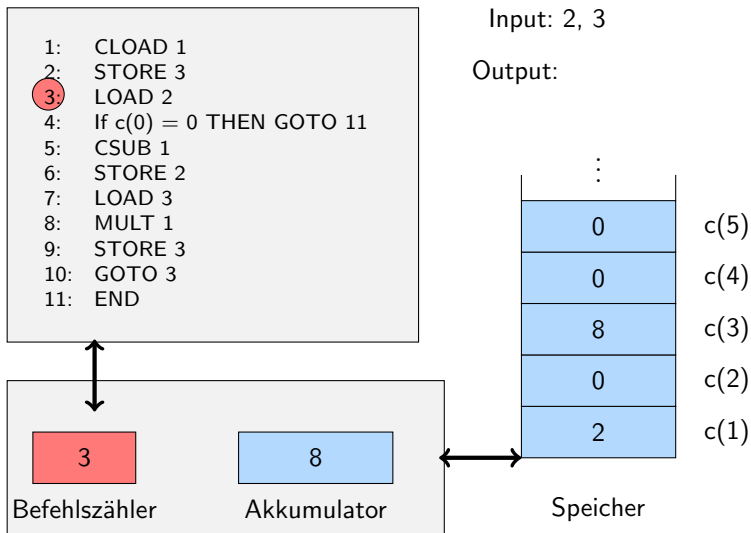
RAM: Beispielprogramm



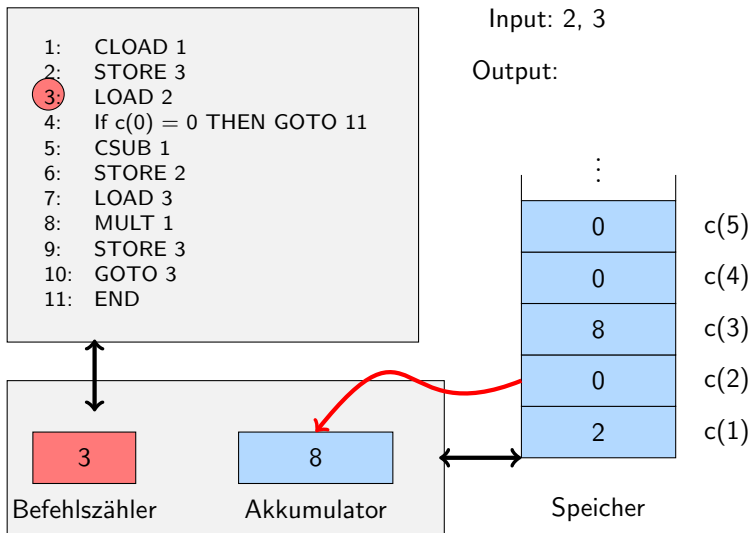
RAM: Beispielprogramm



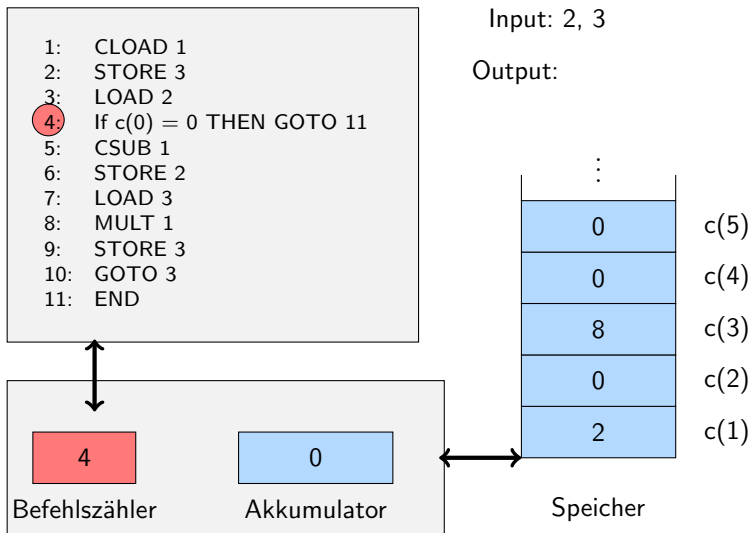
RAM: Beispielprogramm



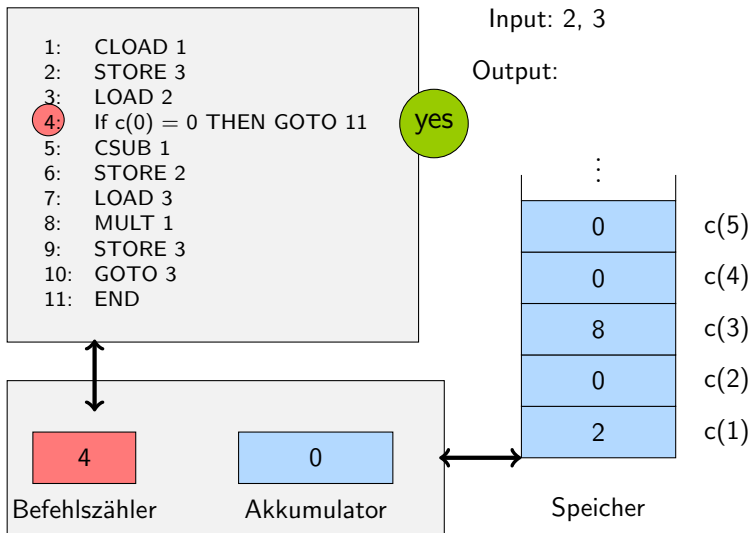
RAM: Beispielprogramm



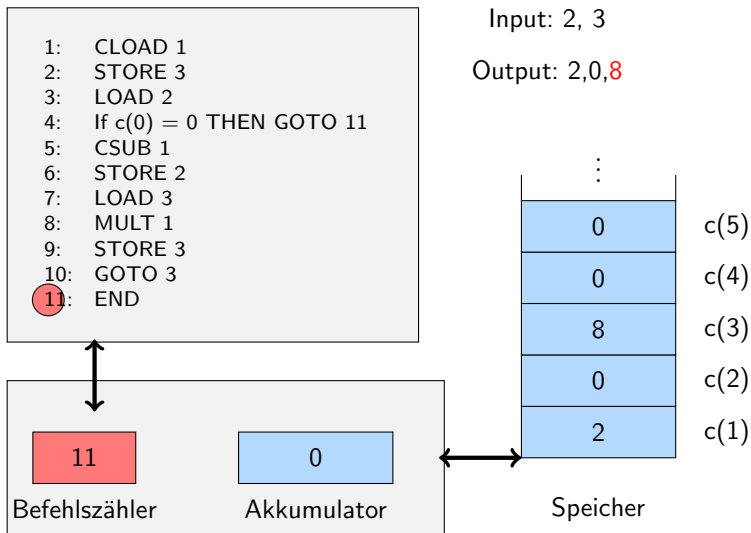
RAM: Beispielprogramm



RAM: Beispielprogramm



RAM: Beispielprogramm



Das Arbeiten mit einer RAM und mit RAM-Programmen ist sehr ähnlich zum Arbeiten mit Assembler-Programmen.

Auf einer RAM können wir alle Befehle realisieren (wie beispielsweise Schleifen und Rekursionen), die wir von höheren Programmiersprachen her gewohnt sind.

Das Arbeiten mit einer RAM und mit RAM-Programmen ist sehr ähnlich zum Arbeiten mit Assembler-Programmen.

Auf einer RAM können wir alle Befehle realisieren (wie beispielsweise Schleifen und Rekursionen), die wir von höheren Programmiersprachen her gewohnt sind.

Modelle für die Rechenzeit

- **Uniformes Kostenmaß:**

Jeder Schritt der RAM zählt als eine Zeiteinheit.

- **Logarithmisches Kostenmaß:**

Die Laufzeitkosten eines Schrittes der RAM sind proportional zur binären Länge der Zahlen in den angesprochenen Registern.

Simulation von RAM durch TM

Satz

Für jede im logarithmischen Kostenmaß $t(n)$ -zeitbeschränkte RAM R gibt es ein Polynom q und eine $q(n + t(n))$ -zeitbeschränkte TM M , die R simuliert.

Satz

Für jede im logarithmischen Kostenmaß $t(n)$ -zeitbeschränkte RAM R gibt es ein Polynom q und eine $q(n + t(n))$ -zeitbeschränkte TM M , die R simuliert.

Im Beweis können wir **ohne Beschränkung der Allgemeinheit (o.B.d.A.)** für die Simulation eine 2-Band-TM statt einer 1-Band-TM verwenden. Warum?

Vorbemerkungen (1): Rechnen mit Polynomen

- Es seien $\alpha, \beta, \gamma \in \mathbb{N}$ geeignet gewählte Konstanten.

Vorbemerkungen (1): Rechnen mit Polynomen

- Es seien $\alpha, \beta, \gamma \in \mathbb{N}$ geeignet gewählte Konstanten.
- Wir werden in unserem Beweis zeigen: Die Laufzeit der Simulation der RAM mit Laufzeitschranke $t(n)$ durch eine 2-Band-TM ist durch $t'(n) = \alpha(n + t(n))^\beta$ nach oben beschränkt.

Vorbemerkungen (1): Rechnen mit Polynomen

- Es seien $\alpha, \beta, \gamma \in \mathbb{N}$ geeignet gewählte Konstanten.
- Wir werden in unserem Beweis zeigen: Die Laufzeit der Simulation der RAM mit Laufzeitschranke $t(n)$ durch eine 2-Band-TM ist durch $t'(n) = \alpha(n + t(n))^\beta$ nach oben beschränkt.
- Die 2-Band-TM mit Laufzeitschranke $t'(n)$ kann nun wiederum mit quadratischem Zeitverlust durch eine 1-Band-TM simuliert werden, also mit einer Laufzeitschranke der Form $t''(n) = \gamma(t'(n))^2$.

Vorbemerkungen (1): Rechnen mit Polynomen

- Es seien $\alpha, \beta, \gamma \in \mathbb{N}$ geeignet gewählte Konstanten.
- Wir werden in unserem Beweis zeigen: Die Laufzeit der Simulation der RAM mit Laufzeitschranke $t(n)$ durch eine 2-Band-TM ist durch $t'(n) = \alpha(n + t(n))^\beta$ nach oben beschränkt.
- Die 2-Band-TM mit Laufzeitschranke $t'(n)$ kann nun wiederum mit quadratischem Zeitverlust durch eine 1-Band-TM simuliert werden, also mit einer Laufzeitschranke der Form $t''(n) = \gamma(t'(n))^2$.
- Für die Simulation der RAM auf der 1-Band-TM ergibt sich somit eine Laufzeitschranke von

$$t''(n) = \gamma(t'(n))^2$$

Vorbemerkungen (1): Rechnen mit Polynomen

- Es seien $\alpha, \beta, \gamma \in \mathbb{N}$ geeignet gewählte Konstanten.
- Wir werden in unserem Beweis zeigen: Die Laufzeit der Simulation der RAM mit Laufzeitschranke $t(n)$ durch eine 2-Band-TM ist durch $t'(n) = \alpha(n + t(n))^\beta$ nach oben beschränkt.
- Die 2-Band-TM mit Laufzeitschranke $t'(n)$ kann nun wiederum mit quadratischem Zeitverlust durch eine 1-Band-TM simuliert werden, also mit einer Laufzeitschranke der Form $t''(n) = \gamma(t'(n))^2$.
- Für die Simulation der RAM auf der 1-Band-TM ergibt sich somit eine Laufzeitschranke von

$$t''(n) = \gamma(t'(n))^2 = \gamma(\alpha(n + t(n))^\beta)^2$$

Vorbemerkungen (1): Rechnen mit Polynomen

- Es seien $\alpha, \beta, \gamma \in \mathbb{N}$ geeignet gewählte Konstanten.
- Wir werden in unserem Beweis zeigen: Die Laufzeit der Simulation der RAM mit Laufzeitschranke $t(n)$ durch eine 2-Band-TM ist durch $t'(n) = \alpha(n + t(n))^\beta$ nach oben beschränkt.
- Die 2-Band-TM mit Laufzeitschranke $t'(n)$ kann nun wiederum mit quadratischem Zeitverlust durch eine 1-Band-TM simuliert werden, also mit einer Laufzeitschranke der Form $t''(n) = \gamma(t'(n))^2$.
- Für die Simulation der RAM auf der 1-Band-TM ergibt sich somit eine Laufzeitschranke von

$$t''(n) = \gamma(t'(n))^2 = \gamma(\alpha(n + t(n))^\beta)^2 = \gamma\alpha^2 \cdot (n + t(n))^{2\beta}.$$

Vorbemerkungen (1): Rechnen mit Polynomen

- Es seien $\alpha, \beta, \gamma \in \mathbb{N}$ geeignet gewählte Konstanten.
- Wir werden in unserem Beweis zeigen: Die Laufzeit der Simulation der RAM mit Laufzeitschranke $t(n)$ durch eine 2-Band-TM ist durch $t'(n) = \alpha(n + t(n))^\beta$ nach oben beschränkt.
- Die 2-Band-TM mit Laufzeitschranke $t'(n)$ kann nun wiederum mit quadratischem Zeitverlust durch eine 1-Band-TM simuliert werden, also mit einer Laufzeitschranke der Form $t''(n) = \gamma(t'(n))^2$.
- Für die Simulation der RAM auf der 1-Band-TM ergibt sich somit eine Laufzeitschranke von

$$t''(n) = \gamma(t'(n))^2 = \gamma(\alpha(n + t(n))^\beta)^2 = \gamma\alpha^2 \cdot (n + t(n))^{2\beta}.$$

- Diese Laufzeitschranke ist polynomiell in $n + t(n)$, weil sowohl der Term $\gamma\alpha^2$ als auch der Term 2β konstant sind.

Vorbemerkungen (2): Rechnen mit Polynomen

Beobachtung

Die Klasse der Polynome ist unter Hintereinanderausführung abgeschlossen.

Mit anderen Worten:

Wenn sowohl die Abbildung $x \mapsto p(x)$ als auch die Abbildung $x \mapsto q(x)$ Polynome sind, dann ist auch die Abbildung $x \mapsto q(p(x))$ ein Polynom.

Vorbemerkungen (2): Rechnen mit Polynomen

Beobachtung

Die Klasse der Polynome ist unter Hintereinanderausführung abgeschlossen.

Mit anderen Worten:

Wenn sowohl die Abbildung $x \mapsto p(x)$ als auch die Abbildung $x \mapsto q(x)$ Polynome sind, dann ist auch die Abbildung $x \mapsto q(p(x))$ ein Polynom.

Deshalb können wir eine **konstante Anzahl** von Simulationen, deren Zeitverlust jeweils polynomiell nach oben beschränkt ist, ineinander schachteln und erhalten dadurch wiederum eine Simulation mit polynomiell beschränktem Zeitverlust.

Beweis des Satzes über die Simulation

- Wir verwenden eine 2-Band-TM, die die RAM schrittweise simuliert. Auf Band 1 werden die einzelnen Befehle simuliert, und auf Band 2 wird der Inhalt aller verwendeten Register abgespeichert.

Beweis des Satzes über die Simulation

- Wir verwenden eine 2-Band-TM, die die RAM schrittweise simuliert. Auf Band 1 werden die einzelnen Befehle simuliert, und auf Band 2 wird der Inhalt aller verwendeten Register abgespeichert.
- Das RAM-Programm P bestehe aus p Programmzeilen.
- Für jede Programmzeile schreiben wir ein TM-Unterprogramm. Es sei M_i das Unterprogramm für Programmzeile i , $1 \leq i \leq p$.
- Ausserdem spezifizieren wir ein Unterprogramm M_0 für die Initialisierung der TM, und ein Unterprogramm M_{p+1} für die Aufbereitung der Ausgabe des Ergebnisses.

Abspeichern der RAM-Konfiguration auf der TM:

- Den Befehlszähler kann die TM im Zustand abspeichern, da die Länge des RAM-Programms konstant ist.

Beweis (2)

Abspeichern der RAM-Konfiguration auf der TM:

- Den Befehlszähler kann die TM im Zustand abspeichern, da die Länge des RAM-Programms konstant ist.
- Die Registerinhalte werden wie folgt auf Band 2 abgespeichert:

$$\# \# 0 \# \text{bin}(c(0)) \# \# \text{bin}(i_1) \# \text{bin}(c(i_1)) \# \# \dots$$
$$\dots \# \# \text{bin}(i_m) \# \text{bin}(c(i_m)) \# \# \# ,$$

wobei $0, i_1, \dots, i_m$ die Indizes der benutzten Register sind.

Beweis (2)

Abspeichern der RAM-Konfiguration auf der TM:

- Den Befehlszähler kann die TM im Zustand abspeichern, da die Länge des RAM-Programms konstant ist.
- Die Registerinhalte werden wie folgt auf Band 2 abgespeichert:

$$\#\#0\# \text{bin}(c(0))\#\# \text{bin}(i_1)\# \text{bin}(c(i_1))\#\# \dots$$
$$\dots \#\# \text{bin}(i_m)\# \text{bin}(c(i_m))\#\#\#,$$

wobei $0, i_1, \dots, i_m$ die Indizes der benutzten Register sind.

Beobachtung

Der Platzbedarf auf Band 2 ist durch $O(n + t(n))$ beschränkt, weil die RAM für jedes neue Bit, das sie erzeugt, mindestens eine Zeiteinheit benötigt.

Rechenschritt für Rechenschritt simuliert die TM nun die Konfigurationsveränderungen der RAM.

Dazu ruft die TM jeweils das im Programmzähler b spezifizierte Unterprogramm M_b auf.

Das Unterprogramm M_b

- kopiert den Inhalt der in Programmzeile b angesprochenen Register auf Band 1,
- führt die notwendigen Operationen auf diesen Registerinhalten durch,
- kopiert dann das Ergebnis in das in Zeile b angegebene Register auf Band 2 zurück, und
- aktualisiert zuletzt den Programmzähler b .

Beweis (4)

Laufzeitanalyse:

- Die Initialisierung erfordert Zeit $O(n)$.

Laufzeitanalyse:

- Die Initialisierung erfordert Zeit $O(n)$.
- Alle Unterprogramme haben eine Laufzeit, die polynomiell in der Länge des aktuellen Wortes auf Band 2 beschränkt ist.
Also: Eine Laufzeit polynomiell in $n + t(n)$.

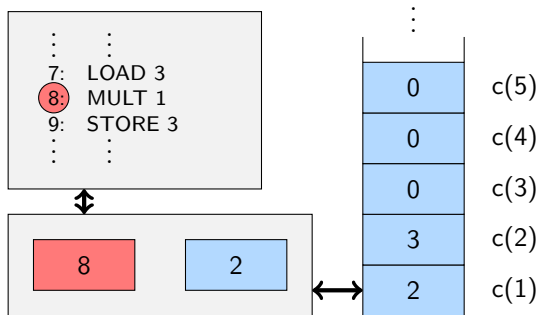
Laufzeitanalyse:

- Die Initialisierung erfordert Zeit $O(n)$.
- Alle Unterprogramme haben eine Laufzeit, die polynomiell in der Länge des aktuellen Wortes auf Band 2 beschränkt ist.
Also: Eine Laufzeit polynomiell in $n + t(n)$.
- Somit ist auch die Gesamtlaufzeit der Simulation polynomiell in $n + t(n)$ beschränkt.

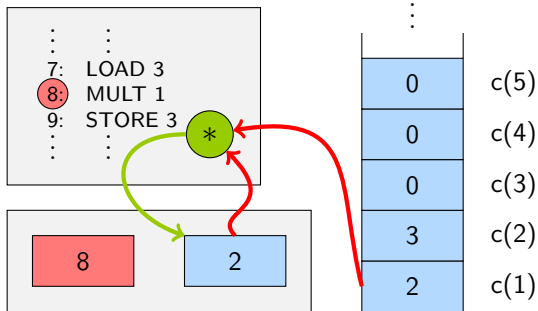
Laufzeitanalyse:

- Die Initialisierung erfordert Zeit $O(n)$.
- Alle Unterprogramme haben eine Laufzeit, die polynomiell in der Länge des aktuellen Wortes auf Band 2 beschränkt ist.
Also: Eine Laufzeit polynomiell in $n + t(n)$.
- Somit ist auch die Gesamtlaufzeit der Simulation polynomiell in $n + t(n)$ beschränkt.
- Ende des Beweises.

Simulierte Registermaschine *M*



Simulierte Registermaschine M



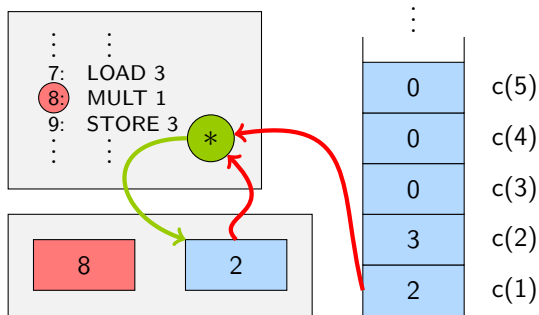
Simulierende Turingmaschine

Programm M_8 : Multiplikation

... | B | **B** | B | B | ...

... | B | **#** | # | 0 | # | 1 | 0 | # | # | 1 | # | 1 | 0 | # | # | 1 | 0 | # | 1 | 1 | # | # | # | B | B | B | ...

Simulierte Registermaschine M



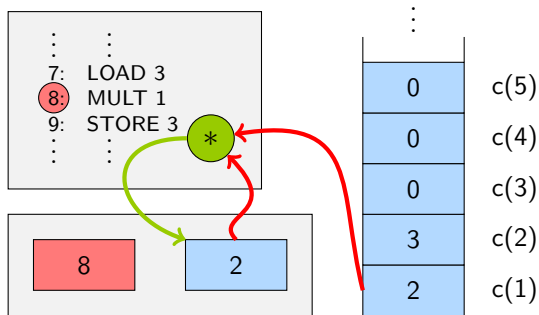
Simulierende Turingmaschine

Kopiere $c(0)$

... B B B B ...

... B # # 0 # 1 0 # # 1 # 1 0 # # 1 0 # 1 1 # # # B B B ...

Simulierte Registermaschine M



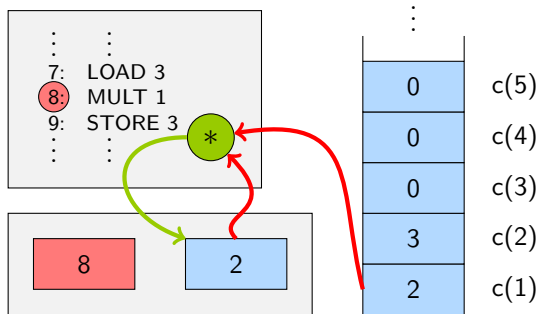
Simulierende Turingmaschine

Kopiere $c(0)$

... B B B B ...

... B # # 0 # 1 0 # # 1 # 1 0 # # 1 0 # 1 1 # # # B B B ...

Simulierte Registermaschine M



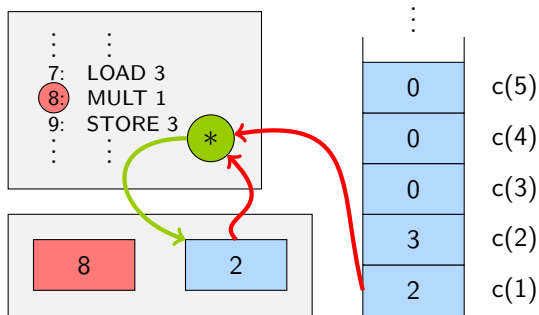
Simulierende Turingmaschine

Kopiere $c(0)$

... B B B B ...

... B # # 0 # 1 0 # # 1 # 1 0 # # 1 0 # 1 1 # # # B B B ...

Simulierte Registermaschine M



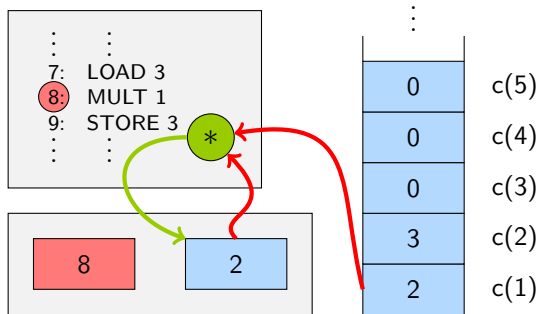
Simulierende Turingmaschine

Kopiere $c(0)$

... B B B B ...

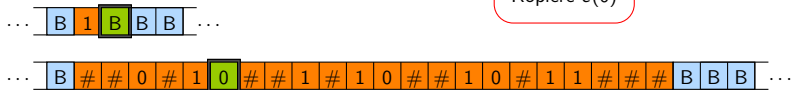
... B # # 0 # 1 0 # # 1 # 1 0 # # 1 0 # 1 1 # # # B B B ...

Simulierte Registermaschine M

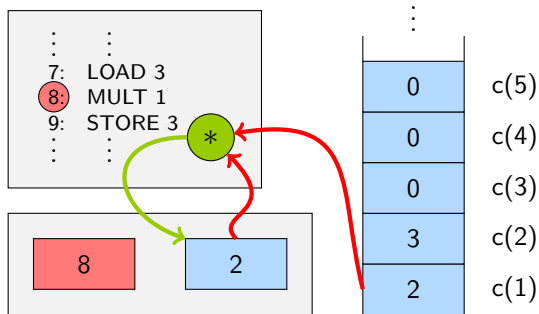


Simulierende Turingmaschine

Kopiere $c(0)$



Simulierte Registermaschine M



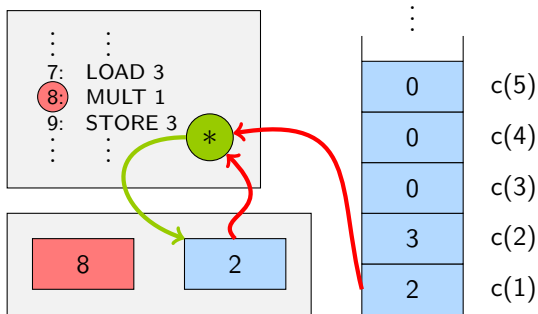
Simulierende Turingmaschine

Kopiere $c(1)$

... B 1 0 B B B ...

... B # # 0 # 1 0 # # 1 # 1 0 # # 1 0 # 1 1 # # # B B B ...

Simulierte Registermaschine M

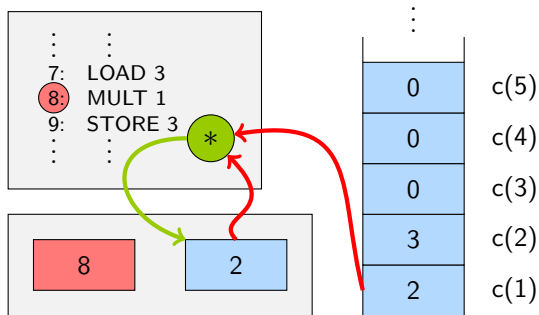


Simulierende Turingmaschine

Kopiere $c(1)$



Simulierte Registermaschine M



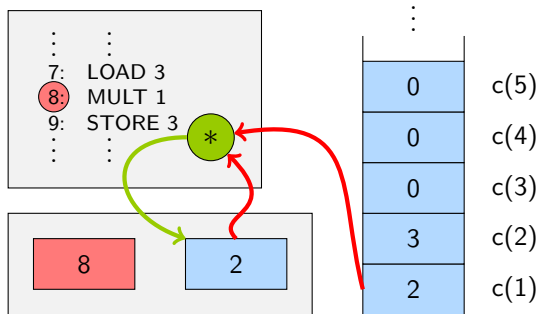
Simulierende Turingmaschine

Kopiere $c(1)$

... B 1 0 # B B B ...

... B # # 0 # 1 0 # # 1 # 1 0 # # 1 0 # 1 1 # # # B B B ...

Simulierte Registermaschine M



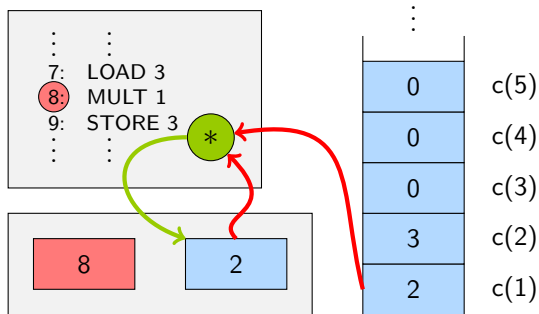
Simulierende Turingmaschine

Kopiere $c(1)$

... B 1 0 # B B B ...

... B # # 0 # 1 0 # # 1 # 1 0 # # 1 0 # 1 1 # # # B B B ...

Simulierte Registermaschine M



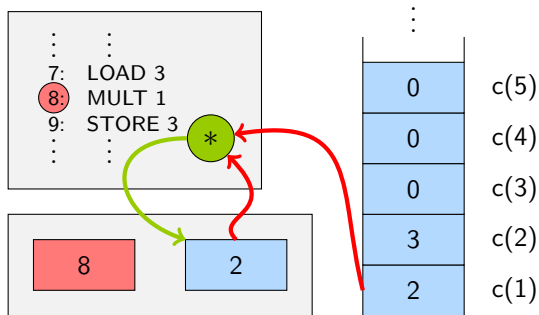
Simulierende Turingmaschine

Kopiere $c(1)$

... B 1 0 # B B B ...

... B # # 0 # 1 0 # # 1 # 1 0 # # 1 0 # 1 1 # # # B B B ...

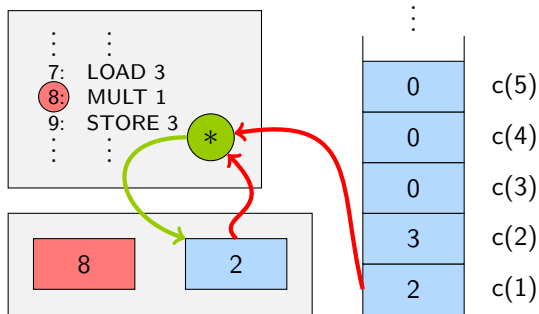
Simulierte Registermaschine M



Simulierende Turingmaschine



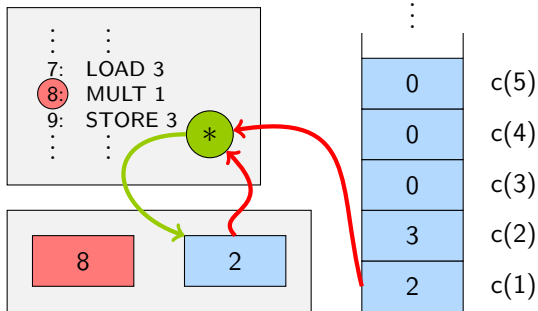
Simulierte Registermaschine M



Simulierende Turingmaschine



Simulierte Registermaschine M

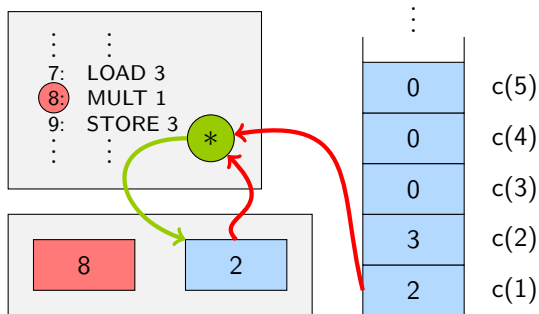


Simulierende Turingmaschine

Multiplizieren



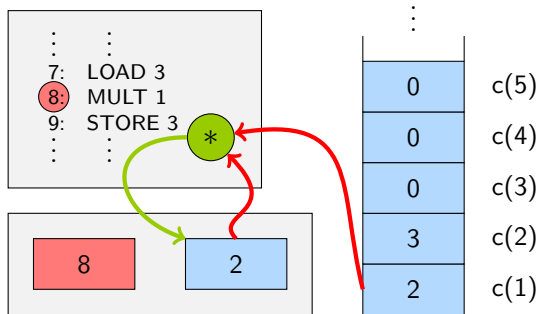
Simulierte Registermaschine M



Simulierende Turingmaschine



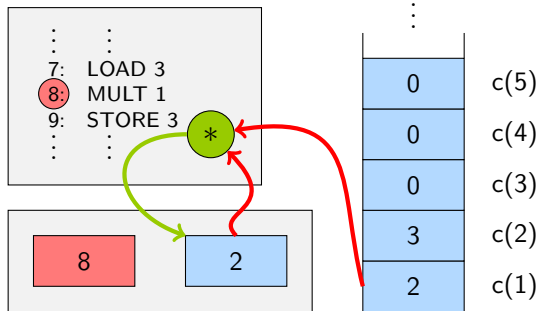
Simulierte Registermaschine M



Simulierende Turingmaschine

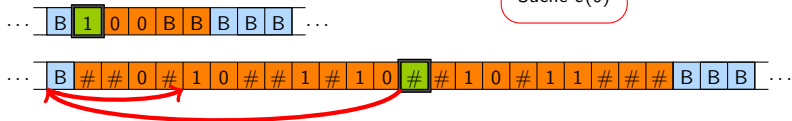


Simulierte Registermaschine M

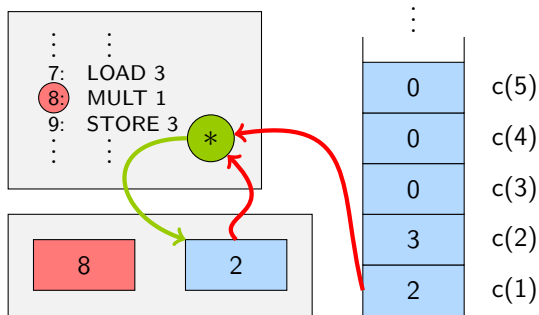


Simulierende Turingmaschine

Suche $c(0)$



Simulierte Registermaschine M



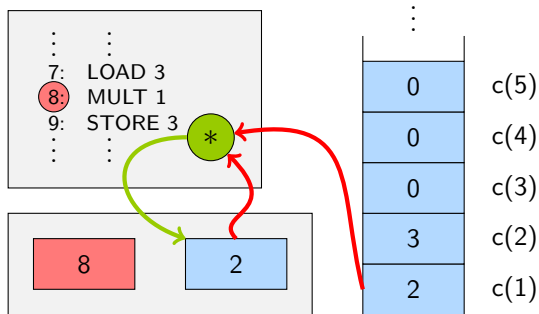
Simulierende Turingmaschine

... B 1 0 0 B B B B B ...

... B # # 0 # 1 0 # # 1 # 1 0 # # 1 0 # 1 1 # # # B B B ...

Update $c(0)$

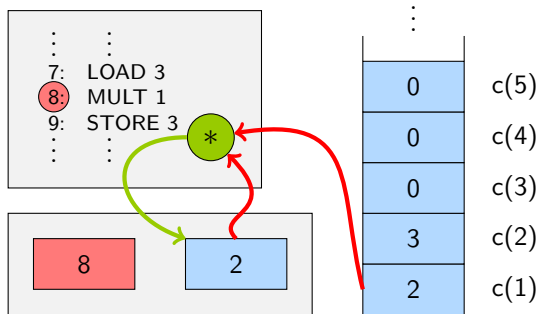
Simulierte Registermaschine M



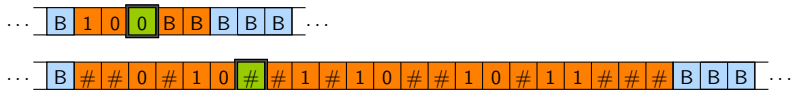
Simulierende Turingmaschine



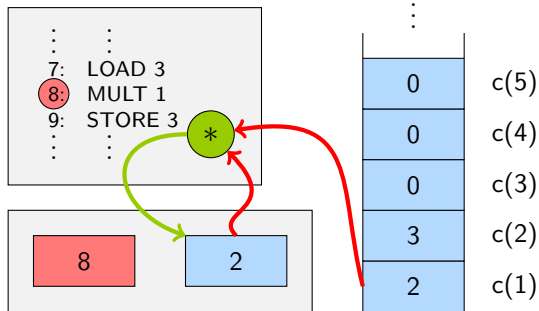
Simulierte Registermaschine M



Simulierende Turingmaschine

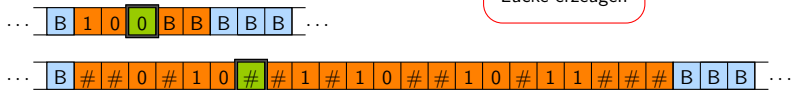


Simulierte Registermaschine M

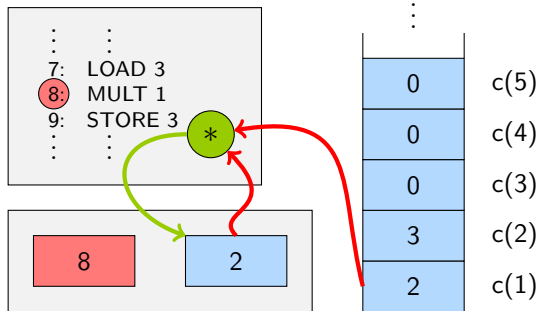


Simulierende Turingmaschine

Lücke erzeugen



Simulierte Registermaschine M



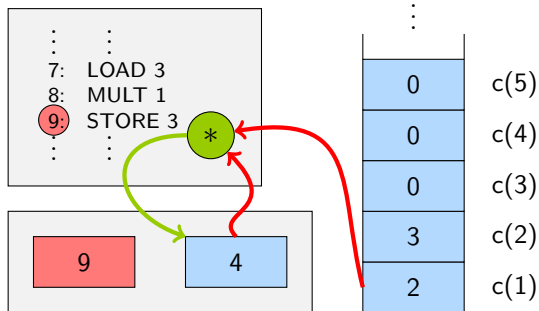
Simulierende Turingmaschine

Lücke erzeugen

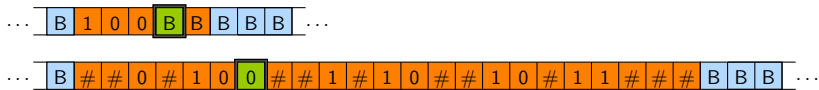
... B 1 0 0 B B B B B ...

... B # # 0 # 1 0 # # 1 # 1 0 # # 1 0 # 1 1 # # # B B B ...

Simulierte Registermaschine M



Simulierende Turingmaschine



Simulation von TM durch RAM

Satz

Jede $t(n)$ -zeitbeschränkte TM kann durch eine RAM simuliert werden, die zeitbeschränkt ist durch

- $O(t(n) + n)$ im uniformen Kostenmaß und
- $O((t(n) + n) \cdot \log(t(n) + n))$ im logarithmischen Kostenmaß.

Beweis (1)

- Wir nehmen an, dass es sich um eine TM mit einseitig beschränktem Band handelt, dessen Zellen mit $0, 1, 2, 3, 4, 5, \dots$ durchnummeriert sind. (Übungsaufgabe: Rechtfertigen Sie diese Annahme.)
- Die Zustände in Q und die Buchstaben im Bandalphabet Γ werden durchnummeriert und mit ihren Nummern identifiziert, so dass sie in den Registern abgespeichert werden können.
- Das Blank B erhält dabei die Nummer 0 .

Beweis (1)

- Wir nehmen an, dass es sich um eine TM mit einseitig beschränktem Band handelt, dessen Zellen mit $0, 1, 2, 3, 4, 5, \dots$ durchnummeriert sind. (Übungsaufgabe: Rechtfertigen Sie diese Annahme.)
- Die Zustände in Q und die Buchstaben im Bandalphabet Γ werden durchnummeriert und mit ihren Nummern identifiziert, so dass sie in den Registern abgespeichert werden können.
- Das Blank B erhält dabei die Nummer 0 .
- Das Register 1 speichert den Index der Kopfposition.
- Das Register 2 speichert den aktuellen Zustand.
- Die Register $3, 4, 5, 6, 7, 8, \dots$ speichern die Inhalte der Bandzellen $0, 1, 2, 3, 4, 5, \dots$

Beweis (2)

Die TM wird nun Schritt für Schritt durch die RAM simuliert.

Auswahl des richtigen TM-Übergangs

Die RAM verwendet eine zweistufige if-Abfrage:

- Auf einer ersten Stufe von $|Q|$ vielen if-Abfragen wird der aktuelle Zustand selektiert.
- Für jeden möglichen Zustand gibt es dann eine zweite Stufe von $|\Gamma|$ vielen if-Abfragen, die das gelesene Zeichen selektieren.

Beweis (2)

Die TM wird nun Schritt für Schritt durch die RAM simuliert.

Auswahl des richtigen TM-Übergangs

Die RAM verwendet eine zweistufige if-Abfrage:

- Auf einer ersten Stufe von $|Q|$ vielen if-Abfragen wird der aktuelle Zustand selektiert.
- Für jeden möglichen Zustand gibt es dann eine zweite Stufe von $|\Gamma|$ vielen if-Abfragen, die das gelesene Zeichen selektieren.

Durchführung des TM-Übergangs

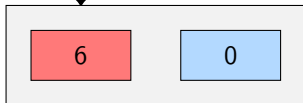
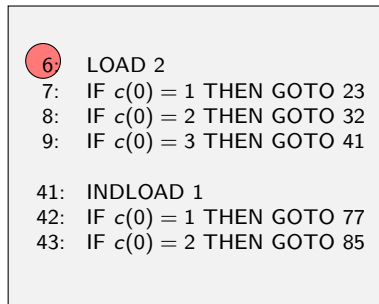
Je nach Ausgang der if-Abfragen aktualisiert die RAM

- den TM-Zustand in Register 2,
- den Inhalt der TM-Bandzelle in Register $c(1)$ und
- die TM-Kopfposition in Register 1.

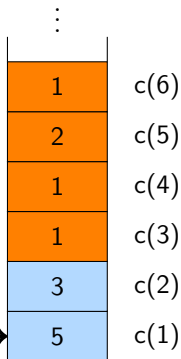
Simulierte Turingmaschine M



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



Simulierende Registermaschine



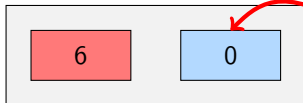
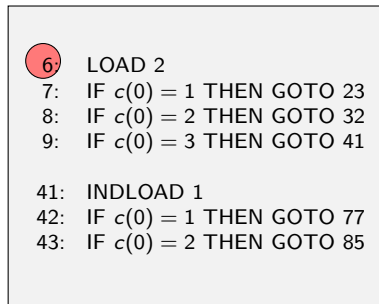
Zustand

Kopfposition

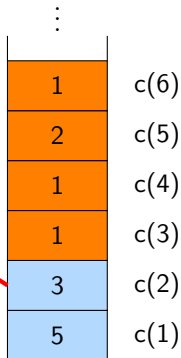
Simulierte Turingmaschine M



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



Simulierende Registermaschine



Zustand

Kopfposition

Simulierte Turingmaschine M



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



```

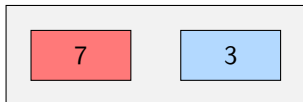
6:  LOAD 2
7:  IF c(0) = 1 THEN GOTO 23
8:  IF c(0) = 2 THEN GOTO 32
9:  IF c(0) = 3 THEN GOTO 41

```

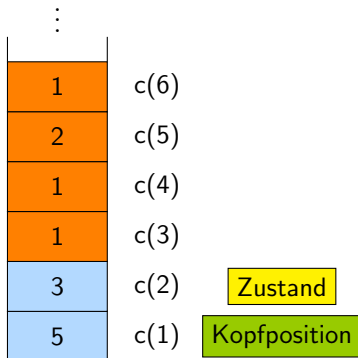
```

41:  INDLOAD 1
42:  IF c(0) = 1 THEN GOTO 77
43:  IF c(0) = 2 THEN GOTO 85

```



Simulierende Registermaschine



Simulierte Turingmaschine M



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



```

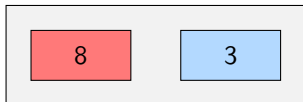
6:  LOAD 2
7:  IF c(0) = 1 THEN GOTO 23
8:  IF c(0) = 2 THEN GOTO 32
9:  IF c(0) = 3 THEN GOTO 41

```

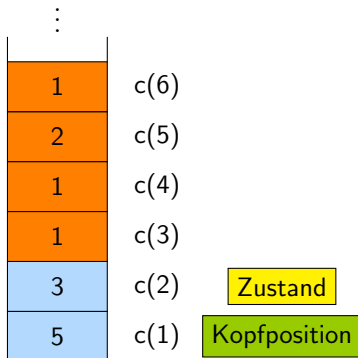
```

41: INDLOAD 1
42: IF c(0) = 1 THEN GOTO 77
43: IF c(0) = 2 THEN GOTO 85

```



Simulierende Registermaschine



Simulierte Turingmaschine M

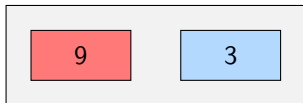


δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	

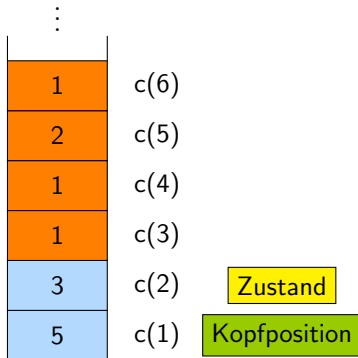


```

6:  LOAD 2
7:  IF c(0) = 1 THEN GOTO 23
8:  IF c(0) = 2 THEN GOTO 32
9:  IF c(0) = 3 THEN GOTO 41
41: INDLOAD 1
42: IF c(0) = 1 THEN GOTO 77
43: IF c(0) = 2 THEN GOTO 85
    
```



Simulierende Registermaschine



Simulierte Turingmaschine M



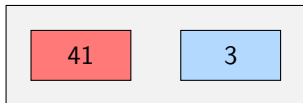
δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



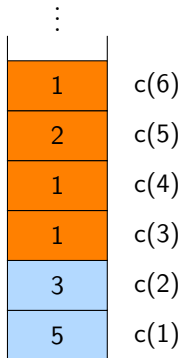
```

6:  LOAD 2
7:  IF c(0) = 1 THEN GOTO 23
8:  IF c(0) = 2 THEN GOTO 32
9:  IF c(0) = 3 THEN GOTO 41
41: INDLOAD 1
42: IF c(0) = 1 THEN GOTO 77
43: IF c(0) = 2 THEN GOTO 85
    
```

41



Simulierende Registermaschine



Zustand

Kopfposition

Simulierte Turingmaschine M



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	

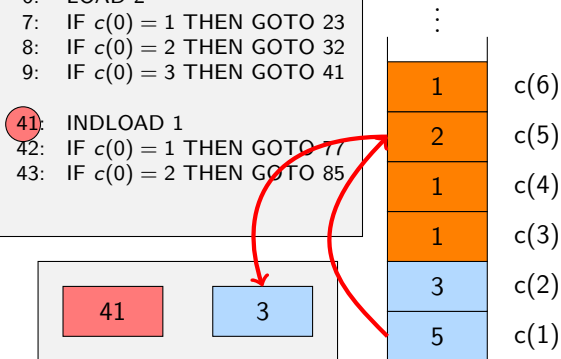


```

6:  LOAD 2
7:  IF c(0) = 1 THEN GOTO 23
8:  IF c(0) = 2 THEN GOTO 32
9:  IF c(0) = 3 THEN GOTO 41

41: INDLOAD 1
42: IF c(0) = 1 THEN GOTO 77
43: IF c(0) = 2 THEN GOTO 85
    
```

Simulierende Registermaschine



Zustand

Kopfposition

Simulierte Turingmaschine M



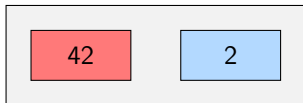
δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



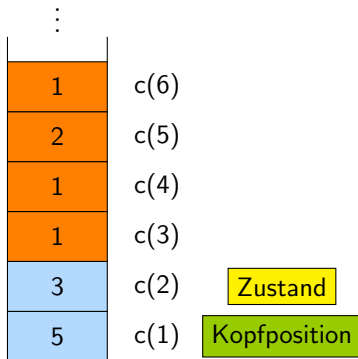
```

6:  LOAD 2
7:  IF c(0) = 1 THEN GOTO 23
8:  IF c(0) = 2 THEN GOTO 32
9:  IF c(0) = 3 THEN GOTO 41

41:  INDLOAD 1
42:  IF c(0) = 1 THEN GOTO 77
43:  IF c(0) = 2 THEN GOTO 85
    
```



Simulierende Registermaschine



Simulierte Turingmaschine M



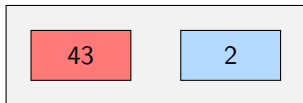
δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



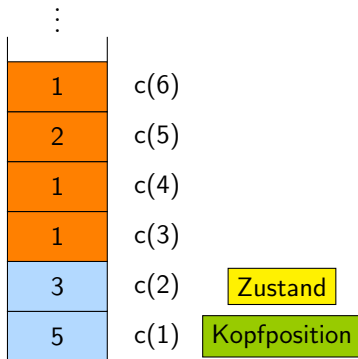
```

6:  LOAD 2
7:  IF c(0) = 1 THEN GOTO 23
8:  IF c(0) = 2 THEN GOTO 32
9:  IF c(0) = 3 THEN GOTO 41

41:  INDLOAD 1
42:  IF c(0) = 1 THEN GOTO 77
43:  IF c(0) = 2 THEN GOTO 85
    
```



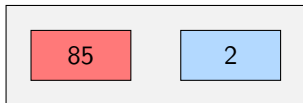
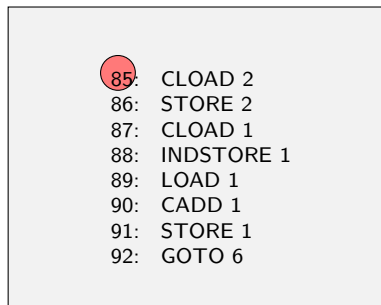
Simulierende Registermaschine



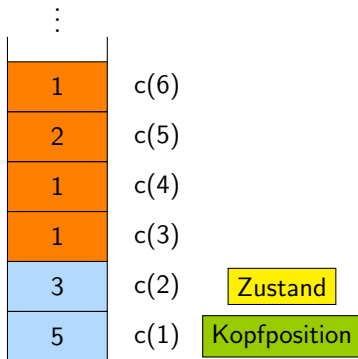
Simulierte Turingmaschine *M*



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



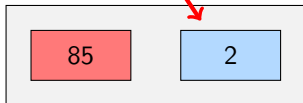
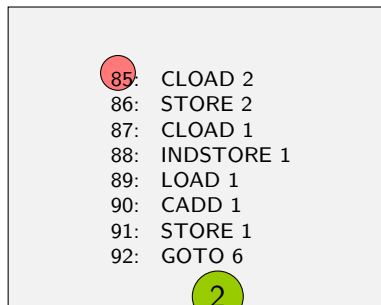
Simulierende Registermaschine



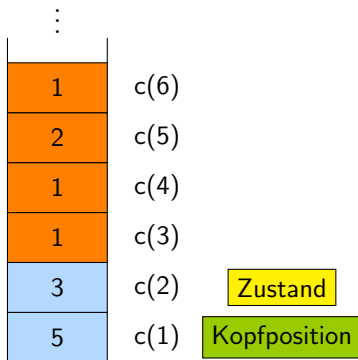
Simulierte Turingmaschine *M*



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



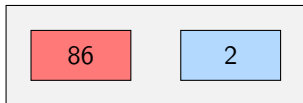
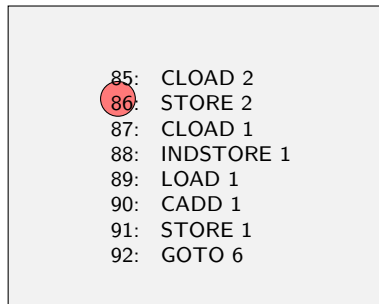
Simulierende Registermaschine



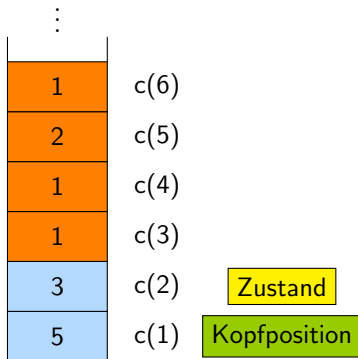
Simulierte Turingmaschine *M*



δ	1	2	<i>B</i>
q_1			
q_2			
q_3		$(q_2, 1, R)$	



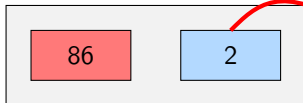
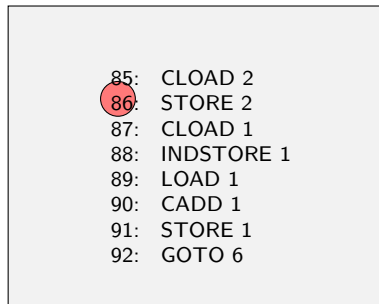
Simulierende Registermaschine



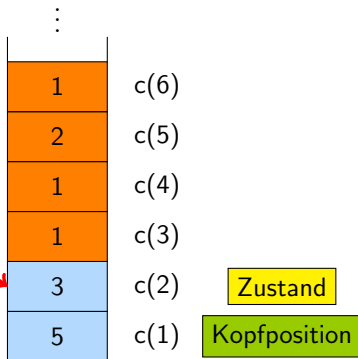
Simulierte Turingmaschine *M*



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



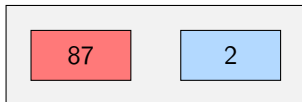
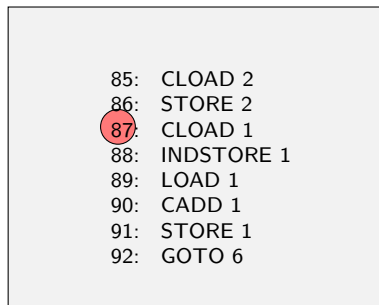
Simulierende Registermaschine



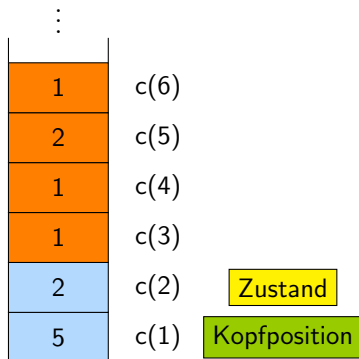
Simulierte Turingmaschine *M*



δ	1	2	<i>B</i>
q_1			
q_2			
q_3		$(q_2, 1, R)$	



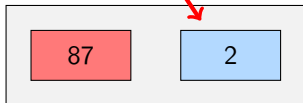
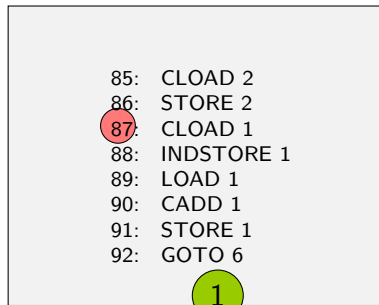
Simulierende Registermaschine



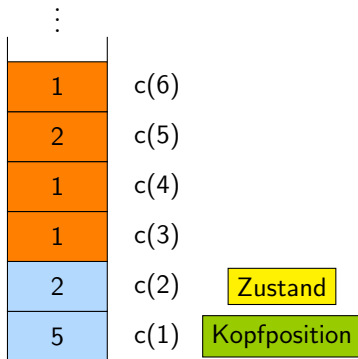
Simulierte Turingmaschine *M*



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



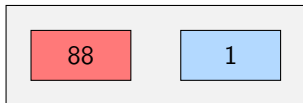
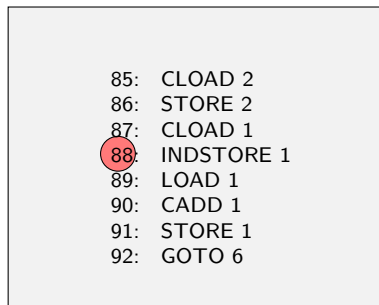
Simulierende Registermaschine



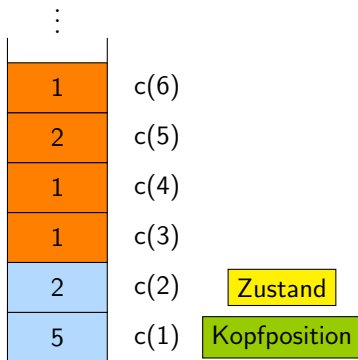
Simulierte Turingmaschine M



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



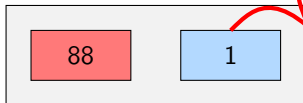
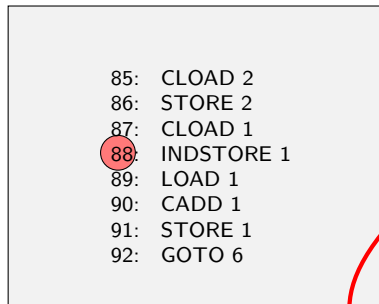
Simulierende Registermaschine



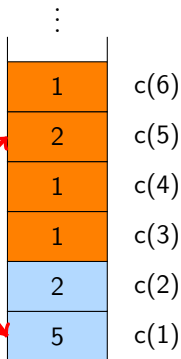
Simulierte Turingmaschine M



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



Simulierende Registermaschine



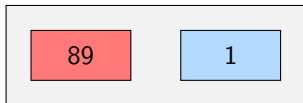
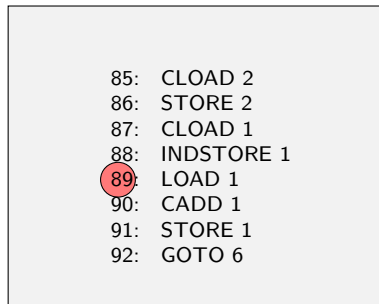
Zustand

Kopfposition

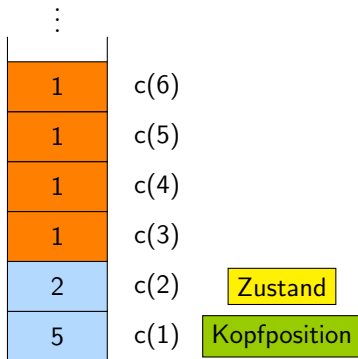
Simulierte Turingmaschine M



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



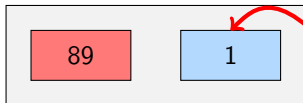
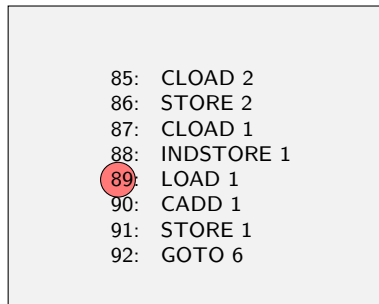
Simulierende Registermaschine



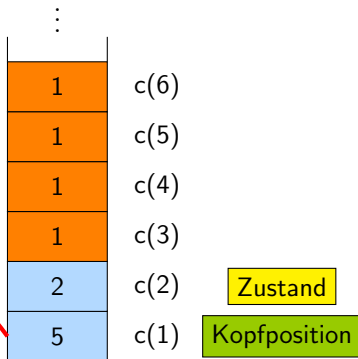
Simulierte Turingmaschine M



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



Simulierende Registermaschine



Simulierte Turingmaschine M

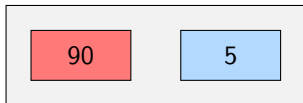


δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	

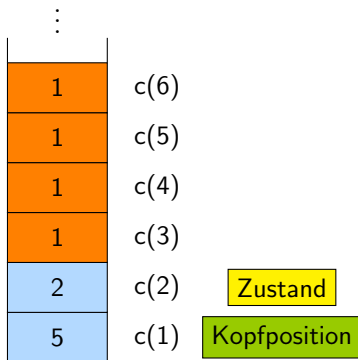


```

85: CLOAD 2
86: STORE 2
87: CLOAD 1
88: INDSTORE 1
89: LOAD 1
90: CADD 1
91: STORE 1
92: GOTO 6
    
```



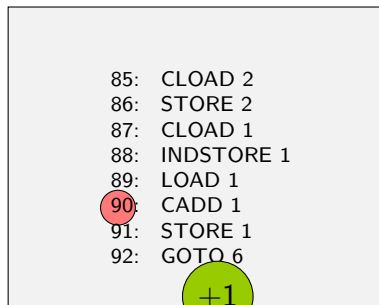
Simulierende Registermaschine



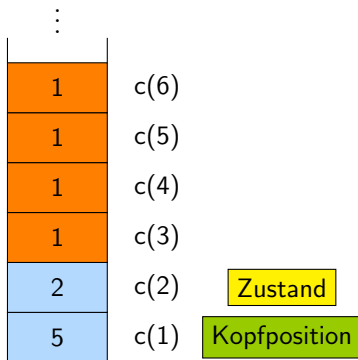
Simulierte Turingmaschine M



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



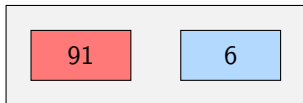
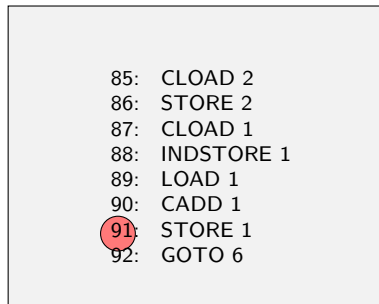
Simulierende Registermaschine



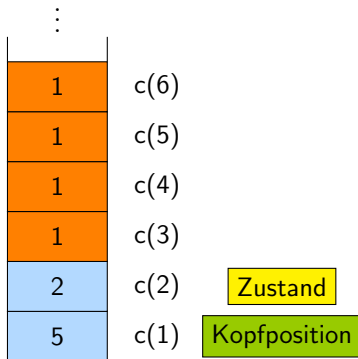
Simulierte Turingmaschine M



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



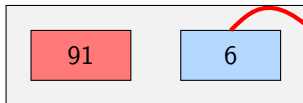
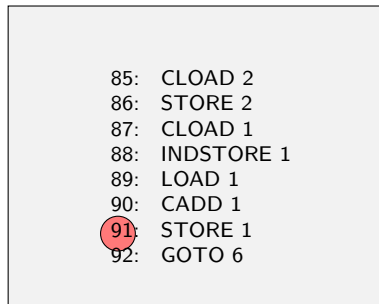
Simulierende Registermaschine



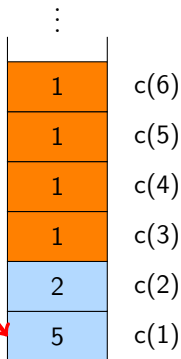
Simulierte Turingmaschine M



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



Simulierende Registermaschine



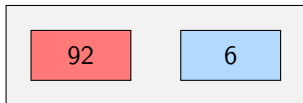
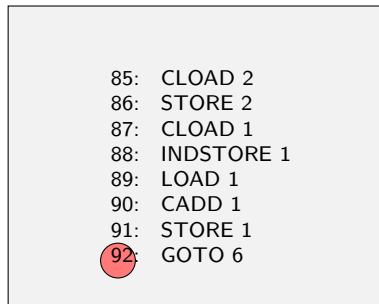
Zustand

Kopfposition

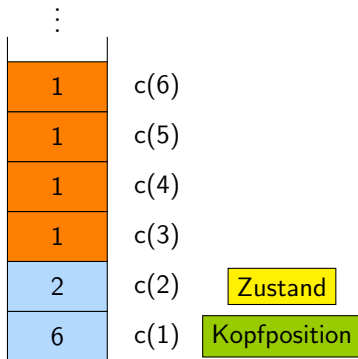
Simulierte Turingmaschine M



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



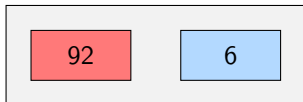
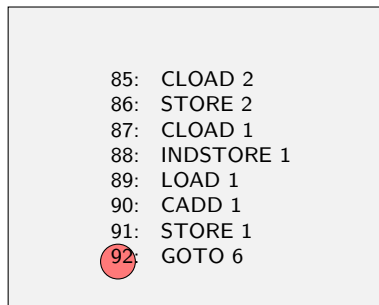
Simulierende Registermaschine



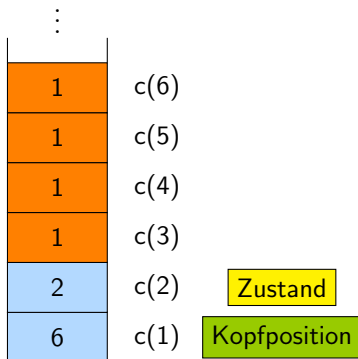
Simulierte Turingmaschine *M*



δ	1	2	<i>B</i>
q_1			
q_2			
q_3		$(q_2, 1, R)$	



Simulierende Registermaschine



Laufzeitanalyse im **uniformen** Kostenmodell:

- Die Initialisierung kann in Zeit $O(n)$ durchgeführt werden.
- Die Simulation jedes einzelnen TM-Schrittes hat konstante Laufzeit.

Laufzeitanalyse im **uniformen** Kostenmodell:

- Die Initialisierung kann in Zeit $O(n)$ durchgeführt werden.
- Die Simulation jedes einzelnen TM-Schrittes hat konstante Laufzeit.
- Insgesamt ist die Simulationszeit somit $O(n + t(n))$.

Laufzeitanalyse im **logarithmischen** Kostenmodell:

- Die in den Registern gespeicherten Zahlen repräsentieren Zustände, Zeichen und Bandpositionen.

Laufzeitanalyse im **logarithmischen** Kostenmodell:

- Die in den Registern gespeicherten Zahlen repräsentieren Zustände, Zeichen und Bandpositionen.
- Zustände und Zeichen haben eine konstante Kodierungslänge.

Laufzeitanalyse im **logarithmischen** Kostenmodell:

- Die in den Registern gespeicherten Zahlen repräsentieren Zustände, Zeichen und Bandpositionen.
- Zustände und Zeichen haben eine konstante Kodierungslänge.
- Die Bandpositionen, die während der Simulation angesprochen werden, sind durch $\max\{n, t(n)\} \leq n + t(n)$ beschränkt. Die Kodierungslänge dieser Positionen ist also $O(\log(t(n) + n))$.

Laufzeitanalyse im **logarithmischen** Kostenmodell:

- Die in den Registern gespeicherten Zahlen repräsentieren Zustände, Zeichen und Bandpositionen.
- Zustände und Zeichen haben eine konstante Kodierungslänge.
- Die Bandpositionen, die während der Simulation angesprochen werden, sind durch $\max\{n, t(n)\} \leq n + t(n)$ beschränkt. Die Kodierungslänge dieser Positionen ist also $O(\log(t(n) + n))$.
- Damit kann die Simulation jedes einzelnen TM-Schrittes in Zeit $O(\log(t(n) + n))$ durchgeführt werden.

Laufzeitanalyse im **logarithmischen** Kostenmodell:

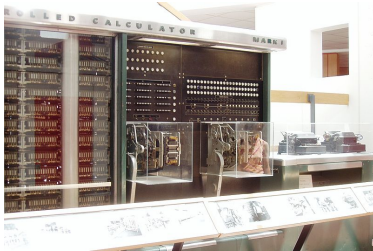
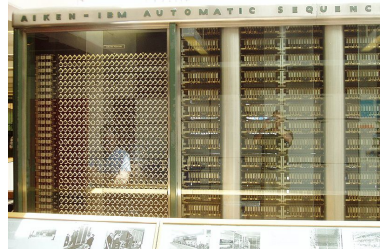
- Die in den Registern gespeicherten Zahlen repräsentieren Zustände, Zeichen und Bandpositionen.
- Zustände und Zeichen haben eine konstante Kodierungslänge.
- Die Bandpositionen, die während der Simulation angesprochen werden, sind durch $\max\{n, t(n)\} \leq n + t(n)$ beschränkt. Die Kodierungslänge dieser Positionen ist also $O(\log(t(n) + n))$.
- Damit kann die Simulation jedes einzelnen TM-Schrittes in Zeit $O(\log(t(n) + n))$ durchgeführt werden.
- Insgesamt ergibt sich somit eine Simulationszeit von $O((t(n) + n) \log(t(n) + n))$.



Zusammenfassung unserer Simulationen

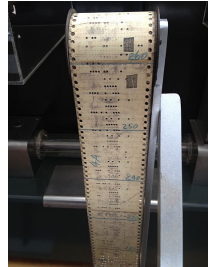
- Die Mehrband-TM kann mit quadratischem Zeitverlust durch eine (1-Band-)TM simuliert werden.
- TM und RAM (im logarithmischen Kostenmodell) können einander gegenseitig mit polynomielltem Zeitverlust simulieren.
- Wenn es also nur um Fragen der Berechenbarkeit von Problemen (oder um ihre **Lösbarkeit in polynomieller Zeit**) geht, so können wir wahlweise auf die TM, die Mehrband-TM oder die RAM zurückgreifen.

Harvard Mark I (1944)



Users:Waldir, Topory/Wikimedia Commons/CC-BY-SA-3.0

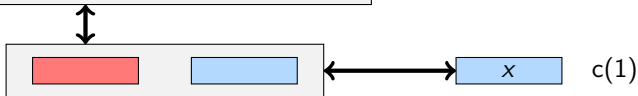
Harvard Mark I (1944)



Arnold Reinhold /CC-BY-SA-3.0

Das Collatz Problem

```
1: LOAD 1
2: IF  $c(0) > 1$  THEN GOTO 4
3: END
4: CADD 1
5: CDIV 2
6: CMULT 2
7: SUB 1
8: IF  $c(0) > 0$  THEN GOTO 13
9: LOAD 1
10: CDIV 2
11: STORE 1
12: GOTO 1
13: LOAD 1
14: CMULT 3
15: CADD 1
16: STORE 1
17: GOTO 1
```

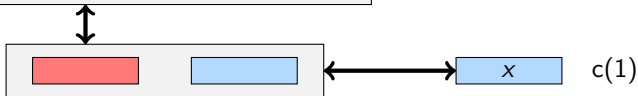



```

1: LOAD 1
2: IF c(0) > 1 THEN GOTO 4
3: END
4: CADD 1
5: CDIV 2
6: CMULT 2
7: SUB 1
8: IF c(0) > 0 THEN GOTO 13
9: LOAD 1
10: CDIV 2
11: STORE 1
12: GOTO 1
13: LOAD 1
14: CMULT 3
15: CADD 1
16: STORE 1
17: GOTO 1

```

$$x \leftarrow \begin{cases} x/2 & \text{wenn } x \text{ gerade} \\ 3x + 1 & \text{wenn } x \text{ ungerade} \end{cases}$$



Das Collatz Problem (1)

$$x \leftarrow \begin{cases} x/2 & \text{wenn } x \text{ gerade} \\ 3x + 1 & \text{wenn } x \text{ ungerade} \end{cases}$$

Das Collatz Problem (1)

$$x \leftarrow \begin{cases} x/2 & \text{wenn } x \text{ gerade} \\ 3x + 1 & \text{wenn } x \text{ ungerade} \end{cases}$$

Mit dieser Iterationsgleichung erhält man z.B. die Zahlenfolgen

- 1, 4, 2, 1, ...

Das Collatz Problem (1)

$$x \leftarrow \begin{cases} x/2 & \text{wenn } x \text{ gerade} \\ 3x + 1 & \text{wenn } x \text{ ungerade} \end{cases}$$

Mit dieser Iterationsgleichung erhält man z.B. die Zahlenfolgen

- 1, 4, 2, 1, ...
- 2, 1, 4, 2, 1, ...

Das Collatz Problem (1)

$$x \leftarrow \begin{cases} x/2 & \text{wenn } x \text{ gerade} \\ 3x + 1 & \text{wenn } x \text{ ungerade} \end{cases}$$

Mit dieser Iterationsgleichung erhält man z.B. die Zahlenfolgen

- 1, 4, 2, 1, ...
- 2, 1, 4, 2, 1, ...
- 3, 10, 5, 16, 8, 4, 2, 1, ...

Das Collatz Problem (1)

$$x \leftarrow \begin{cases} x/2 & \text{wenn } x \text{ gerade} \\ 3x + 1 & \text{wenn } x \text{ ungerade} \end{cases}$$

Mit dieser Iterationsgleichung erhält man z.B. die Zahlenfolgen

- 1, 4, 2, 1, ...
- 2, 1, 4, 2, 1, ...
- 3, 10, 5, 16, 8, 4, 2, 1, ...
- 5, 16, 8, 4, 2, 1, 4, 2, 1

Das Collatz Problem (1)

$$x \leftarrow \begin{cases} x/2 & \text{wenn } x \text{ gerade} \\ 3x + 1 & \text{wenn } x \text{ ungerade} \end{cases}$$

Mit dieser Iterationsgleichung erhält man z.B. die Zahlenfolgen

- 1, 4, 2, 1, ...
- 2, 1, 4, 2, 1, ...
- 3, 10, 5, 16, 8, 4, 2, 1, ...
- 5, 16, 8, 4, 2, 1, 4, 2, 1
- 19, 58, 29, 88, 44, 22, 11, 34, 17, 52, 26,
13, 40, 20, 10, 5, 16, 8, 4, 2, 1, 4, 2, 1, 4, 2, 1

Das Collatz Problem (1)

$$x \leftarrow \begin{cases} x/2 & \text{wenn } x \text{ gerade} \\ 3x + 1 & \text{wenn } x \text{ ungerade} \end{cases}$$

Mit dieser Iterationsgleichung erhält man z.B. die Zahlenfolgen

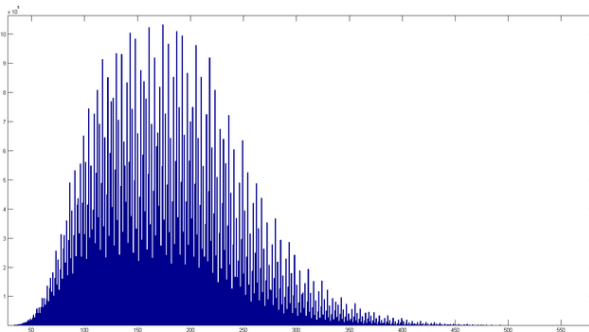
- 1, 4, 2, 1, ...
- 2, 1, 4, 2, 1, ...
- 3, 10, 5, 16, 8, 4, 2, 1, ...
- 5, 16, 8, 4, 2, 1, 4, 2, 1
- 19, 58, 29, 88, 44, 22, 11, 34, 17, 52, 26,
13, 40, 20, 10, 5, 16, 8, 4, 2, 1, 4, 2, 1, 4, 2, 1

Offenes Problem

Hält die obige Registermaschine auf allen Eingaben?

Das Collatz Problem (2)

Statistik der Zahlenfolgenlängen bei Eingaben bis zu 100 Millionen:



User:Allen_McC/Wikimedia
Commons/CC-BY-SA-3.0